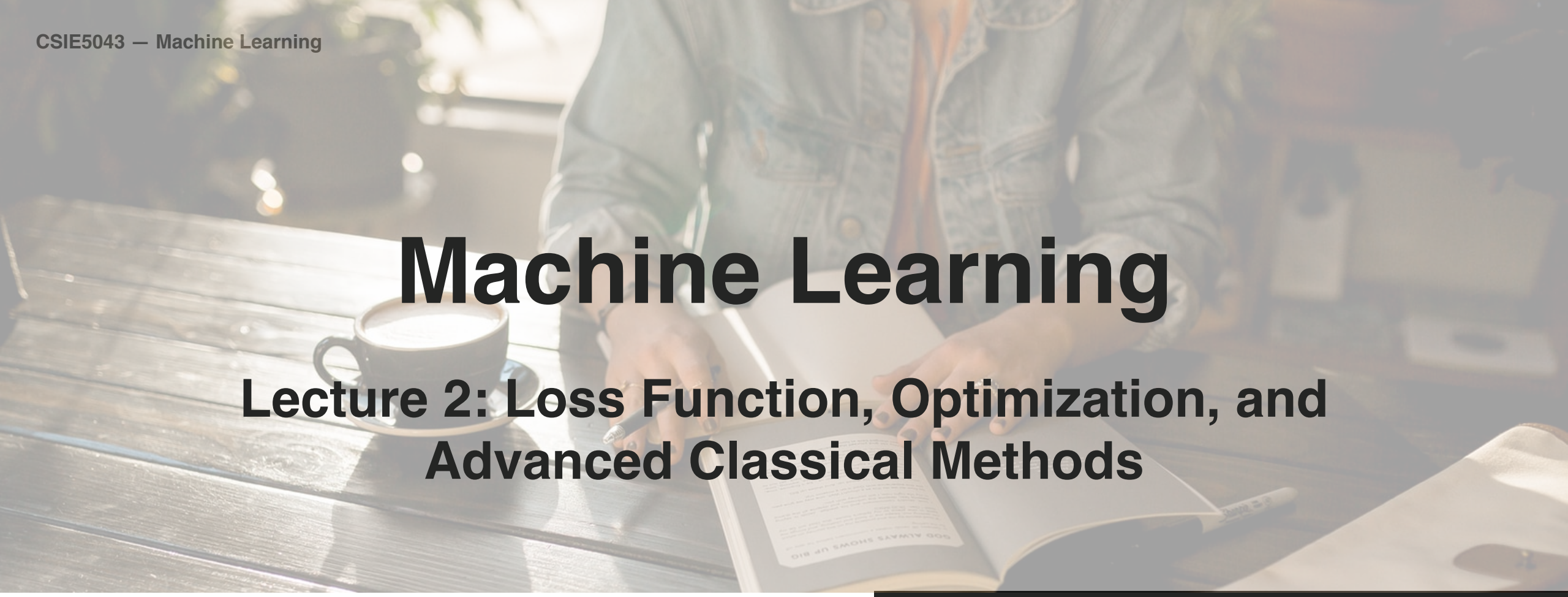


Machine Learning

Lecture 2: Loss Function, Optimization, and Advanced Classical Methods



國立臺灣大學
National Taiwan University



National Taiwan University

Department of Computer Science and Information Engineering

Prof. Chun-Yi Lee

Outline

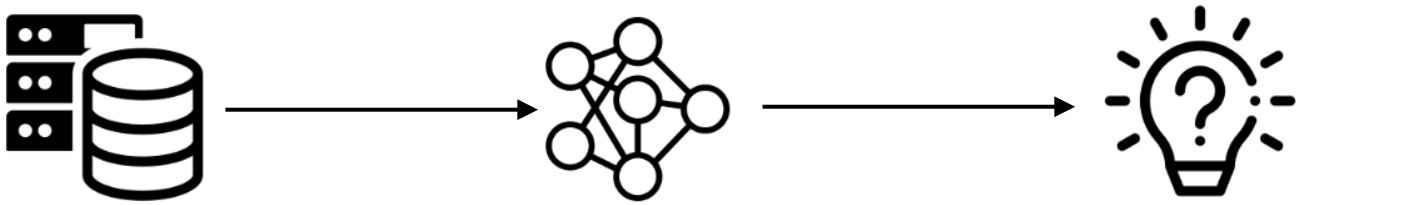
- **Loss function**
- Optimization
- Recap: Basic classification
- Advanced classification



Loss Function

Loss Function Definition

- Function with model parameters:
  - $g(x;\theta), \theta = \{\theta^0, \theta^1, \theta^2, \ldots, \theta^d\}$, d is the data dimension
- A "**good**" function:
  - $g(x_n;\theta) \sim y_n$, $\mathscr{D}$: $\{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$, N is the number of data samples

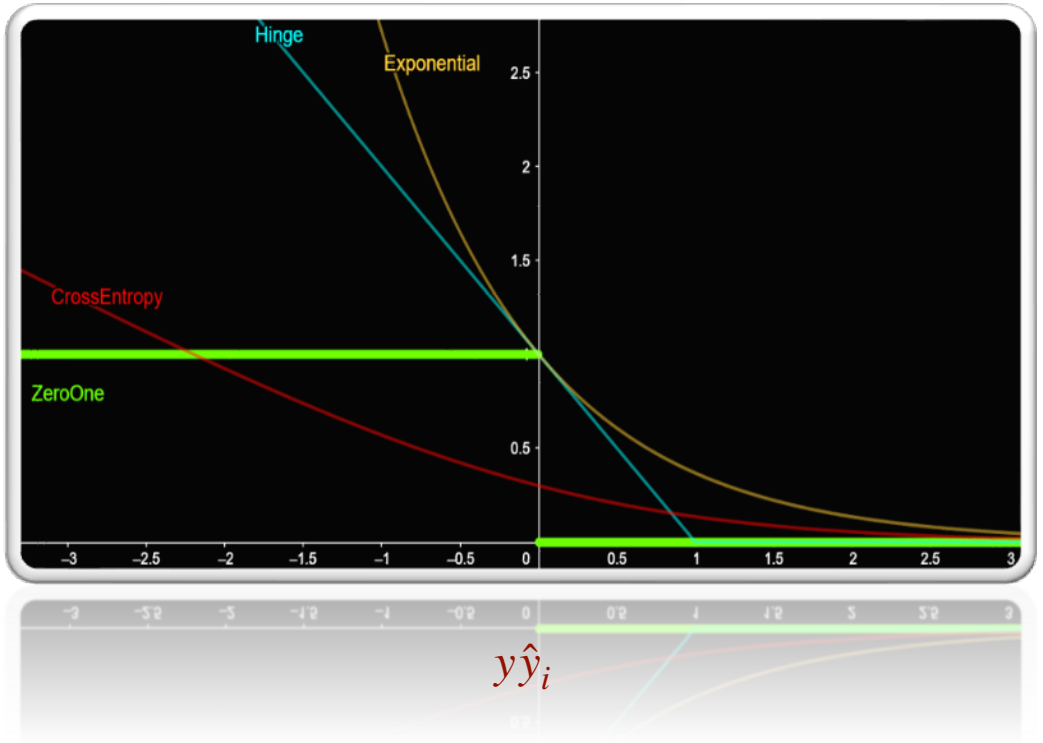


Loss function definition: $\mathscr{L}(\theta) = \mathscr{L}(y, \hat{y} = g(x;\theta)) = \sum_{n=1}^{N} err(g(x_n;\theta), y_n))$

Loss Function

0-1 Loss Function - Binary Classification Metrics

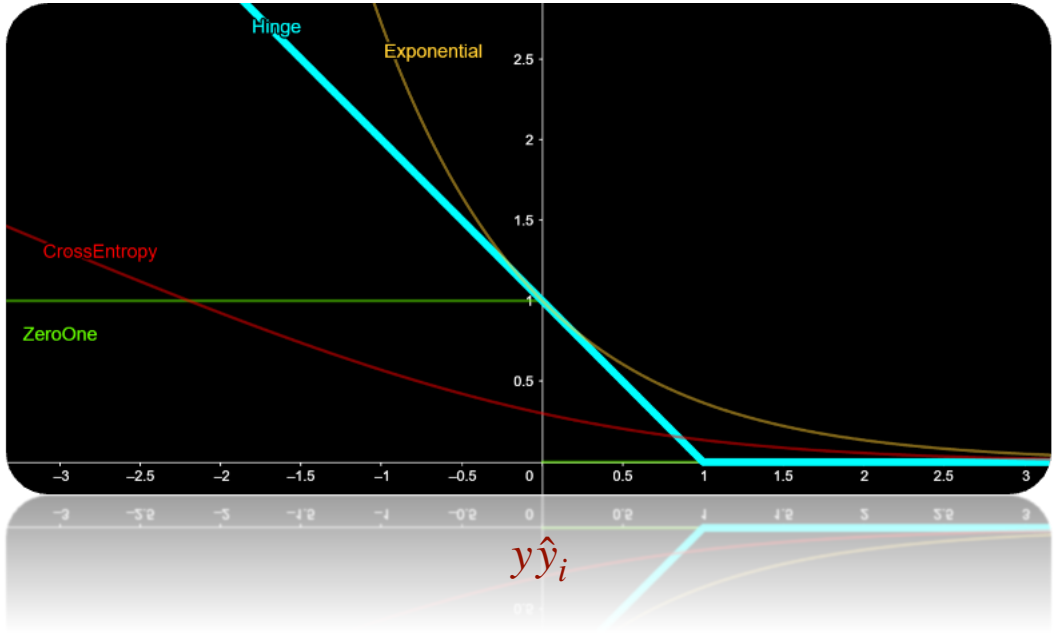
- **0-1 loss** definition:
  - $\mathscr{L}(y_i, \hat{y}_i) = \begin{cases} 1, & y_i \neq \hat{y}_i \\ 0, & y_i = \hat{y}_i \end{cases}$
- **Key properties**:
  - Binary Outcome: outputs **0** (correct) or **1** (wrong)
  - Non-Differentiable: cannot be optimized with gradient-based methods
  - Interpretability: directly counts the number of misclassification
- **Limitations**:
  - Not suitable for training, due to the non-differentiable property



Loss Function

Hinge Loss Function

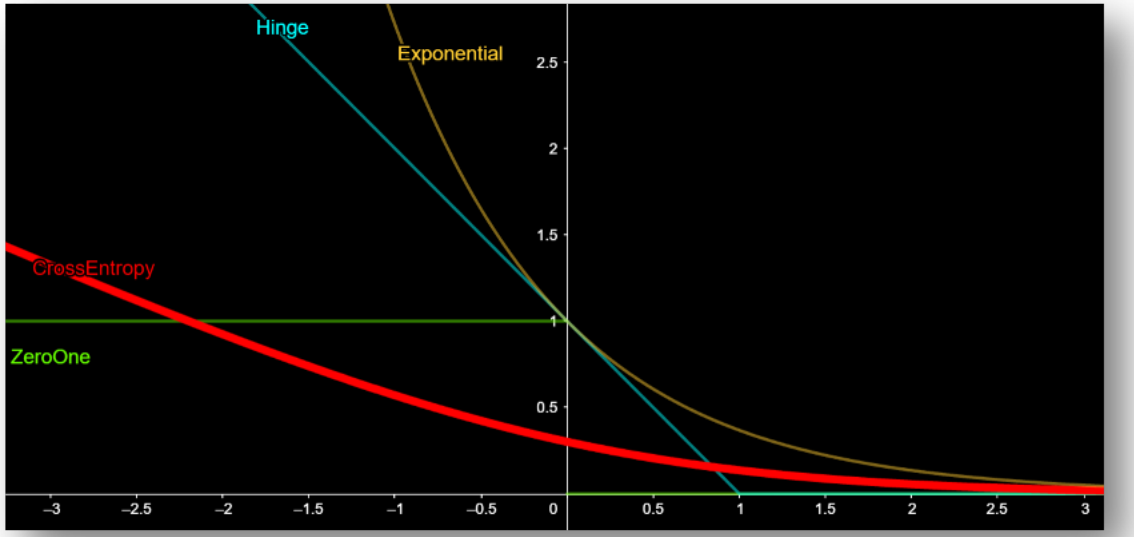
- **Hinge loss** definition:
  - $\mathscr{L}(y_i, \hat{y}_i) = \max(0, 1 - \hat{y}_i y_i)$
  - If $\hat{y}_i y_i < 1$, loss is $1 - \hat{y}_i y_i$
  - If $\hat{y}_i y_i \geq 1$, loss is 0
- **Key properties**:
  - Smoothness: avoid discontinuity problems
  - Convex and piecewise linear: suitable for gradient-based optimization



Loss Function

Cross Entropy - Probabilistic Distribution Involved

- **Cross entropy** definition:
  - $H(p, q) = \mathbb{E}_{x\sim p}[-\log q(x)]$
  - $p(x)$: target distribution (unknown)
  - $q(x)$: predicted distribution (model)
- **Binary cross entropy**
  - $H(p, q) = -p_+ \cdot \log(q_+) - p_- \cdot \log(q_-)$
  - $H(p, q) = -p_+ \cdot \log(q_+) - (1 - p_+) \cdot \log(1 - q_+)$
  - $H(p, q) = -y \cdot \log(q) - (1 - y) \cdot \log(1 - q), \quad y \in \{0,1\}$



$y\hat{y}_i$

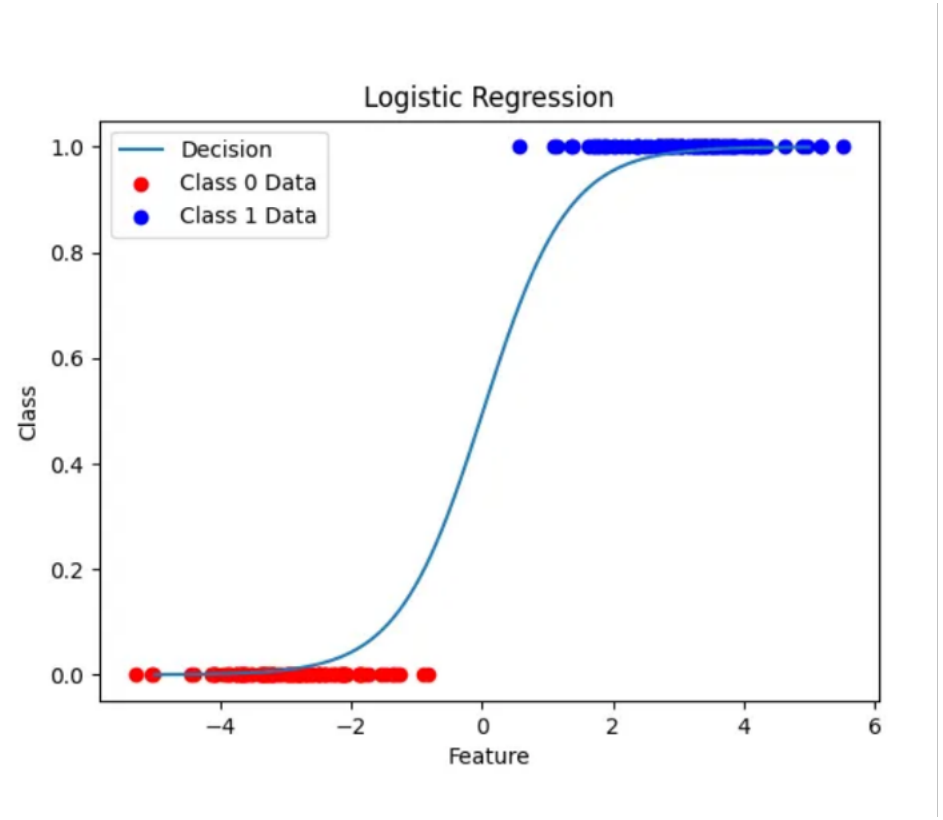
$$H(p, q) = \mathbb{E}_{x\sim p}[-\log q(x)] \quad \geq \quad \mathbb{E}_{x\sim p}[-\log p(x)] = H(p, p) = H(p)$$

Therefore, minimizing $H(p, q)$ forces q to be as close as possible to the true distribution p

Loss Function

Logistic Loss Function - Cross Entropy Loss + Sigmoid Function

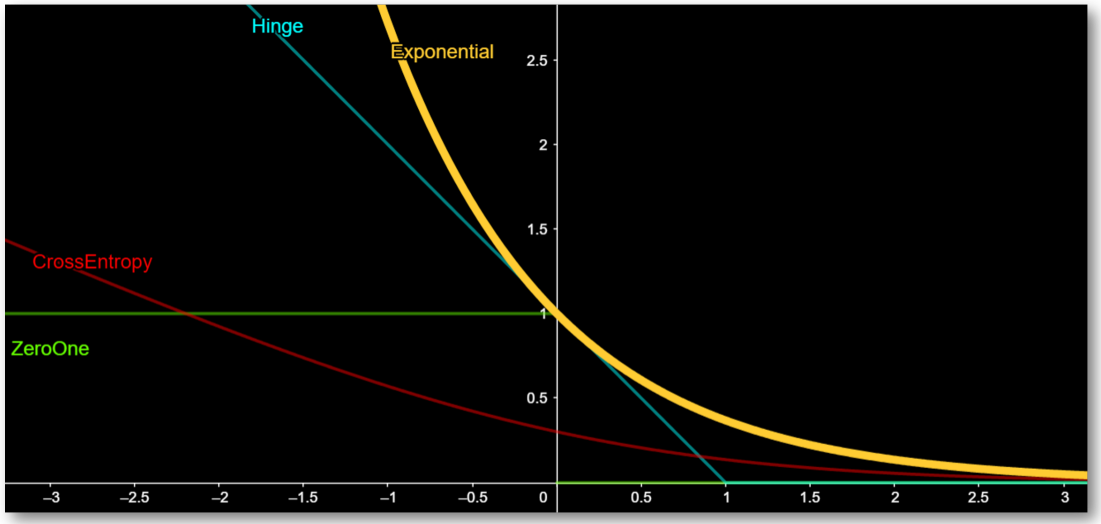
- **Logistic loss definition**:
  - $z_i = \sigma(\hat{y}_i) = \frac{1}{1 + e^{-\hat{y}_i}}$ (**Sigmoid function**)
  - $\mathscr{L}(y_i, z_i) = -y_i \log(z_i) - (1 - y_i)\log(1 - z_i)$
- **Key properties**:
  - Used in logistic regression
  - Binary cross entropy loss with sigmoid link
  - Provides probabilistic outputs
  - Convex: guarantees global optimum for logistic regression
  - Smooth & differentiable: can be optimized through gradient-based methods



Loss Function

Exponential Loss Function - Huge Loss Penalization

- Exponential loss definition:
  - $\mathscr{L}(y_i, \hat{y}_i) = \exp(-\hat{y}_i y_i), \quad y_i \in \{-1, 1\}$
- **Key properties**:
  - Strongly penalizes misclassified samples
  - Even small errors: exponential increases in loss
  - Encourages the model to correct hard-to-classify samples
- **Advantages**:
  - Convex, smooth, and differentiable
- **Disadvantages**:
  - Extremely sensitive to outliers
  - Risk of overfitting noisy data



$y\hat{y}_i$

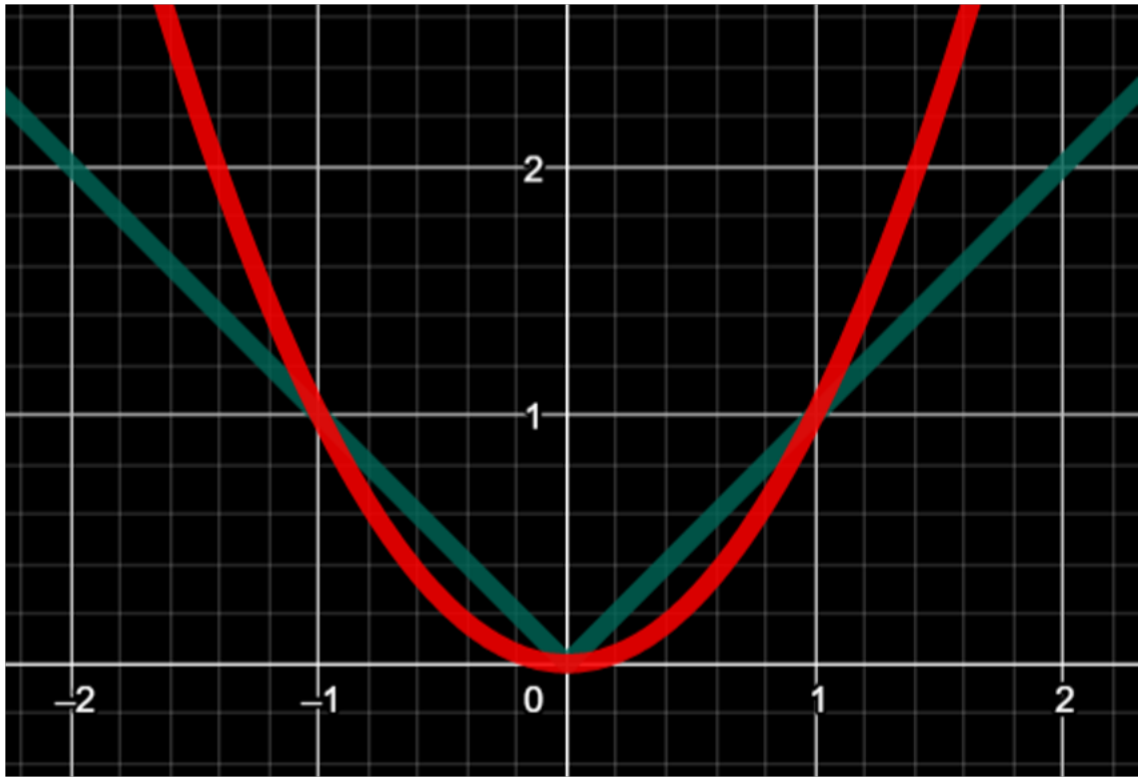
x	-1	-2	-3
Exponential loss	2.718	7.389	20.086
Logistic loss	1.313	2.127	3.049

Loss Function

Basic Loss Function for Regression Task

- **Mean absolute error** (L1 loss / MAE)
  - $\mathscr{L}(y, \hat{y}) = \frac{1}{N} \sum_{n=1}^{N} |y_i - \hat{y}_i|$
  - Measures average absolute deviation
  - Robust to outliers
  - Leads to median as optimal prediction

- **Mean square error** (L2 loss / MSE)
  - $\mathscr{L}(y, \hat{y}) = \frac{1}{N} \sum_{n=1}^{N} (y_i - \hat{y}_i)^2$
  - Sensitive to outliers
  - Leads to mean as optimal prediction



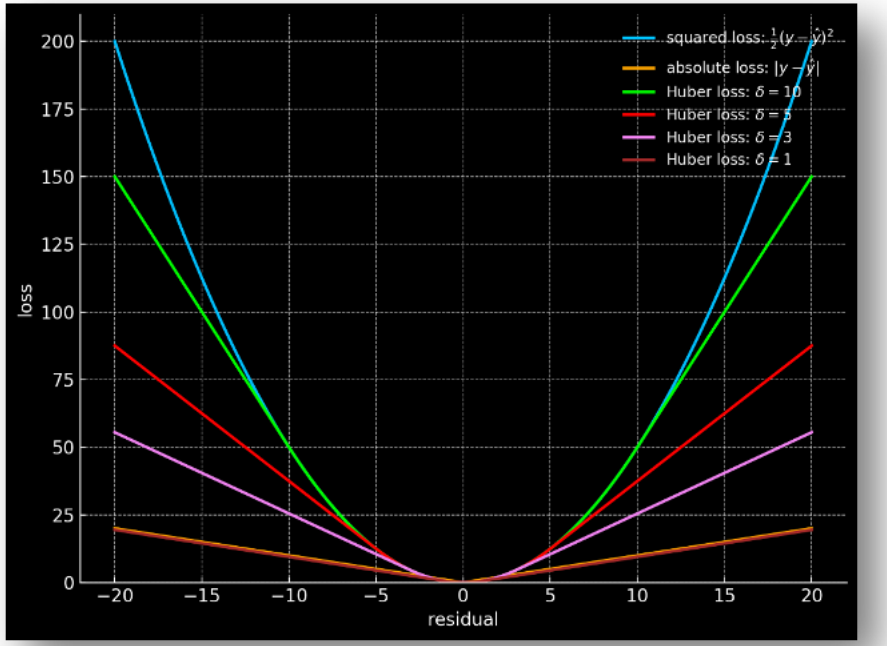
L1 loss

L2 loss

Loss Function

Huber Loss - Combination of L1 and L2 Loss

- **Huber loss definition**:
- $$\mathscr{L}(y,\hat{y}) = \begin{cases} \frac{1}{2}(y-\hat{y})^2, & \text{if } |y-\hat{y}| \le \delta \\ \delta\left(|y-\hat{y}| - \frac{1}{2}\delta\right), & \text{if } |y-\hat{y}| > \delta \end{cases}$$
  - Small errors: behave like squared loss (smooth)
  - Large errors: behave like absolute loss (robust to outliers)

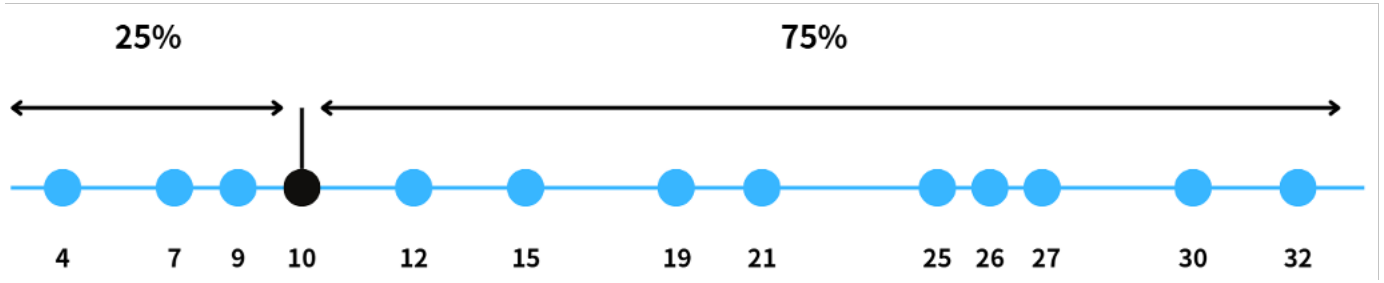


- **Advantages**:
  - More robust to outliers than L2 loss
  - Smooth, and differentiable everywhere (unlike L1 loss)
  - Adjustable δ parameter (controls transition point)

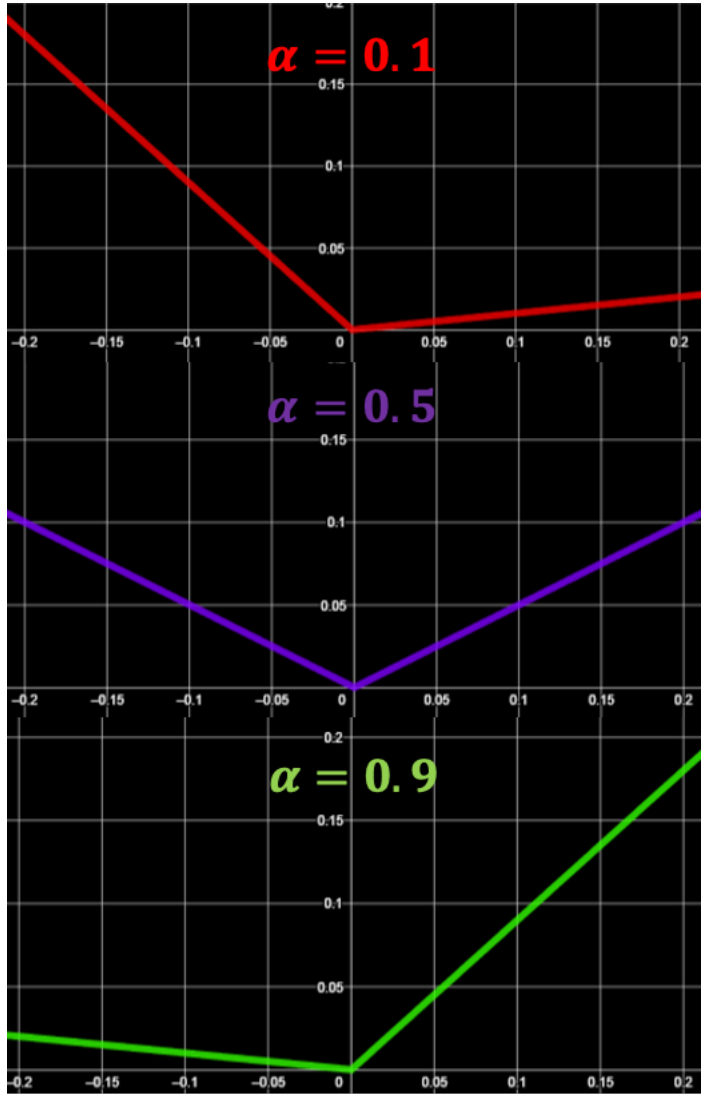
Loss Function

Quantile Loss - Chosen Quantile Prediction of the Target Distribution

- **Quantile loss definition**:
  - $\mathscr{L}(y, \hat{y}) = \begin{cases} \alpha \cdot (y - \hat{y}), & \hat{y} \leq y \\ (1 - \alpha) \cdot (\hat{y} - y), & \hat{y} > y \end{cases}$ Under Estimation / Over Estimation
  - $\alpha = 0.5$: predict the median
  - $\alpha = 0.9$: predict the 90th percentile
- **Advantages**:
  - Models uncertainty and prediction intervals
  - Useful when predicting conditional quantiles
  - Robust to skewed distributions and outliers (at certain quantiles)



Over Estimation $(y - \hat{y}_i)$ Under Estimation



Outline

- Loss function
- **Optimization**
- Recap: Basic classification
- Advanced classification

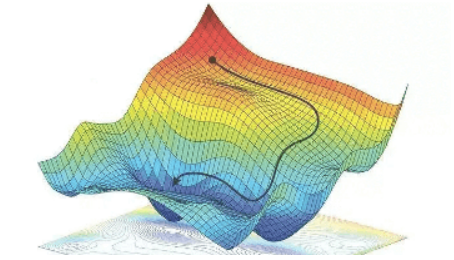


Optimization

Overview of Optimization Methods

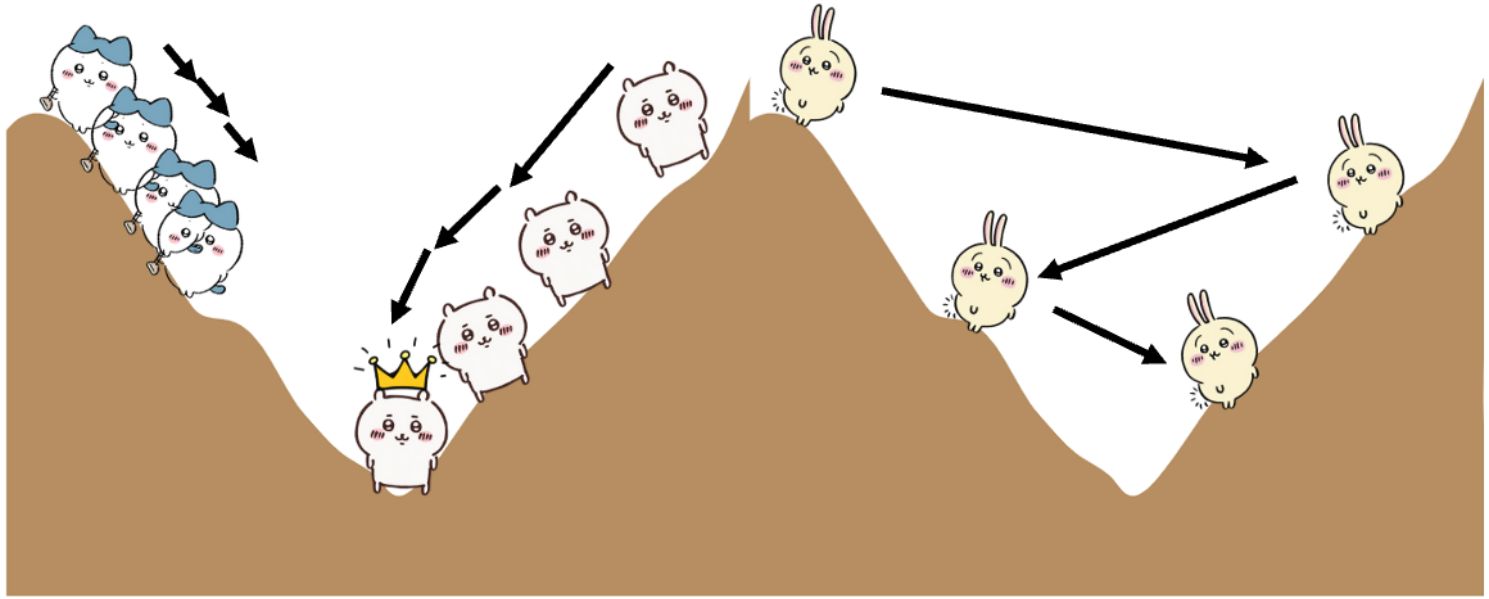
Method	Hessian	Storage cost	Convergence speed
Gradient descent		 $O(d)$	
Newton method		 $O(d^2)$	
Quasi Newton (BFGS)		 $O(d^2)$	
Quasi Newton (L-BFGS)		 $O(md)$	
Conjugate gradient descent		 $O(d)$	

Optimization



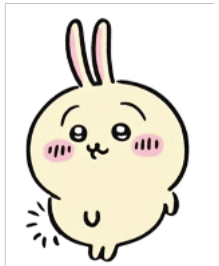
Gradient Descent - 1st Order Iterative Algorithm

- **Gradient descent update rule**
  - $\theta_{t+1} \leftarrow \theta_t - \eta \nabla L(\theta_t)$, $\eta =$ learning rate, $\nabla L(\theta_t) =$ gradient (direction of steepest ascent)
  - **Gradient**: direction of increase, so we move opposite
  - Iteratively update parameters toward loss minimum
  - Final θ is learned parameters





Learning rate η too small
Slow convergence

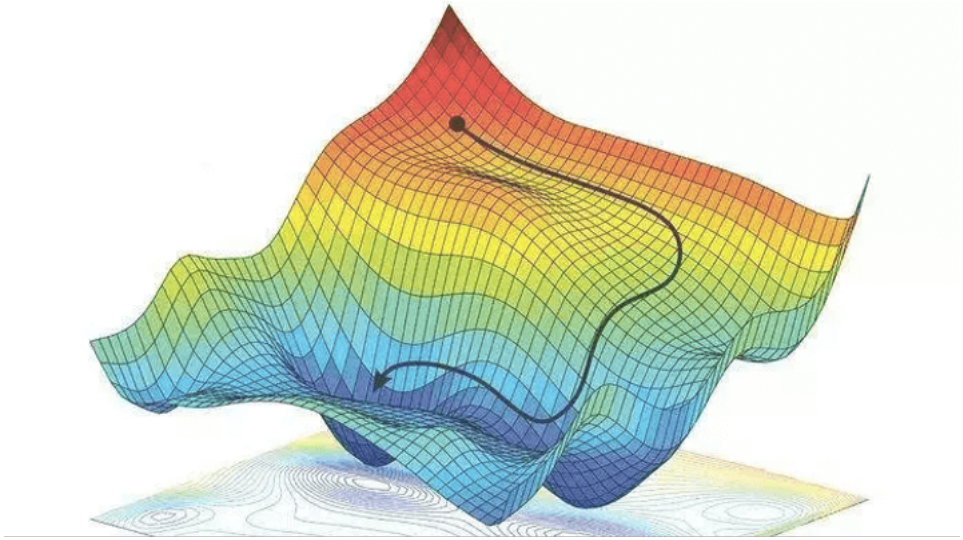


Learning rate η too large
Divergence

Optimization

Gradient Descent - Derivation

- **Goal**: minimize loss function $\mathscr{L}$
  - $\min_{\|v\|=1} \mathscr{L}(\theta_t + \eta v)$
- Taylor expansion (small η)
  - $\mathscr{L}(\theta_t + \eta v) \approx \mathscr{L}(\theta_t) + \eta v^T \nabla \mathscr{L}(\theta_t)$
- Optimization problem
  - $\min_{\|v\|=1} \mathscr{L}(\theta_t) + \eta v^T \nabla \mathscr{L}(\theta_t)$
- Optimal direction (v = opposite direction of $\nabla \mathscr{L}(\theta_t)$)
  - $v = \frac{\nabla \mathscr{L}(\theta_t)}{\|\nabla \mathscr{L}(\theta_t)\|}$



Update rule: $\theta_{t+1} \leftarrow \theta_t - \eta \frac{\nabla \mathscr{L}(\theta_t)}{\|\nabla \mathscr{L}(\theta_t)\|}$, $\theta_{t+1} \leftarrow \theta_t - \tilde{\eta}_t \nabla \mathscr{L}(\theta_t)$, $\tilde{\eta}_t = \frac{\eta}{\|\nabla \mathscr{L}(\theta_t)\|}$

Optimization

Newton Method - 2nd Order Iterative Algorithm

$$\mathscr{L}(\theta_{t+1}) \approx \mathscr{L}(\theta_t) + \nabla\mathscr{L}(\theta_t)^T(\theta_{t+1} - \theta_t) + \frac{1}{2}(\theta_{t+1} - \theta_t)^T H_{\mathscr{L}}(\theta_t)(\theta_{t+1} - \theta_t)$$

$$\frac{d}{d\theta_{t+1}}\mathscr{L}(\theta_{t+1}) = \nabla\mathscr{L}(\theta_t) + H_{\mathscr{L}}(\theta_t)(\theta_{t+1} - \theta_t) = 0$$

- **Newton method update rule**
  - $\theta_{t+1} = \theta_t - H_{\mathscr{L}}(\theta_t)^{-1}\nabla\mathscr{L}(\theta_t)$, H is the Hessian matrix
- Convergence properties:
  - Sensitive to initial guess: good -> fast, poor -> slow
  - Requiring smoothness: continuous 1st and 2nd derivatives
  - Stopping criteria: small parameter change or small gradient
- Complexity characteristics:
  - High per-iteration cost: Jacobian and Hessian matrix
  - Memory intensive: storing Hessian matrix in high dimensions
  - Numerical issues: ill-conditioned Hessian matrix may cause instability

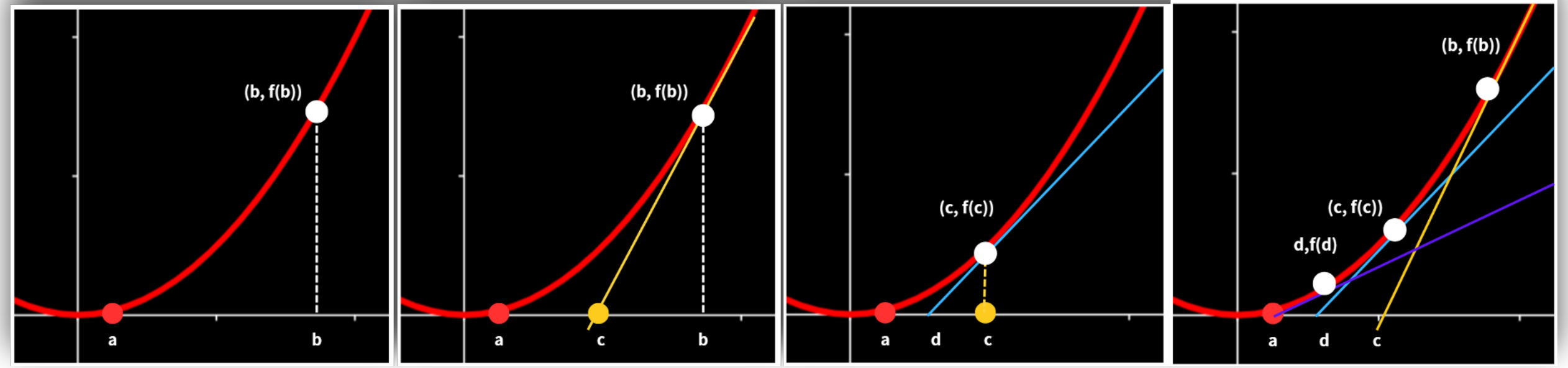
- **Time complexity**
  - Jacobian computation: $O(d)$
  - Hessian computation: $O(d^2)$
  - Hessian computation can be accelerated, if Hessian is sparse or structured

$$H_{\mathscr{L}} = \begin{bmatrix} \frac{\partial^2\mathscr{L}}{\partial(\theta^0)^2} & \frac{\partial^2\mathscr{L}}{\partial\theta^0\partial\theta^1} & \cdots & \frac{\partial^2\mathscr{L}}{\partial\theta^0\partial\theta^d} \\ \frac{\partial^2\mathscr{L}}{\partial\theta^1\partial\theta^0} & \frac{\partial^2\mathscr{L}}{\partial(\theta^1)^2} & \cdots & \frac{\partial^2\mathscr{L}}{\partial\theta^1\partial\theta^d} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2\mathscr{L}}{\partial\theta^d\partial\theta^0} & \frac{\partial^2\mathscr{L}}{\partial\theta^d\partial\theta^1} & \cdots & \frac{\partial^2\mathscr{L}}{\partial(\theta^d)^2} \end{bmatrix}$$

Calculating Hessian is computationally expensive

Optimization

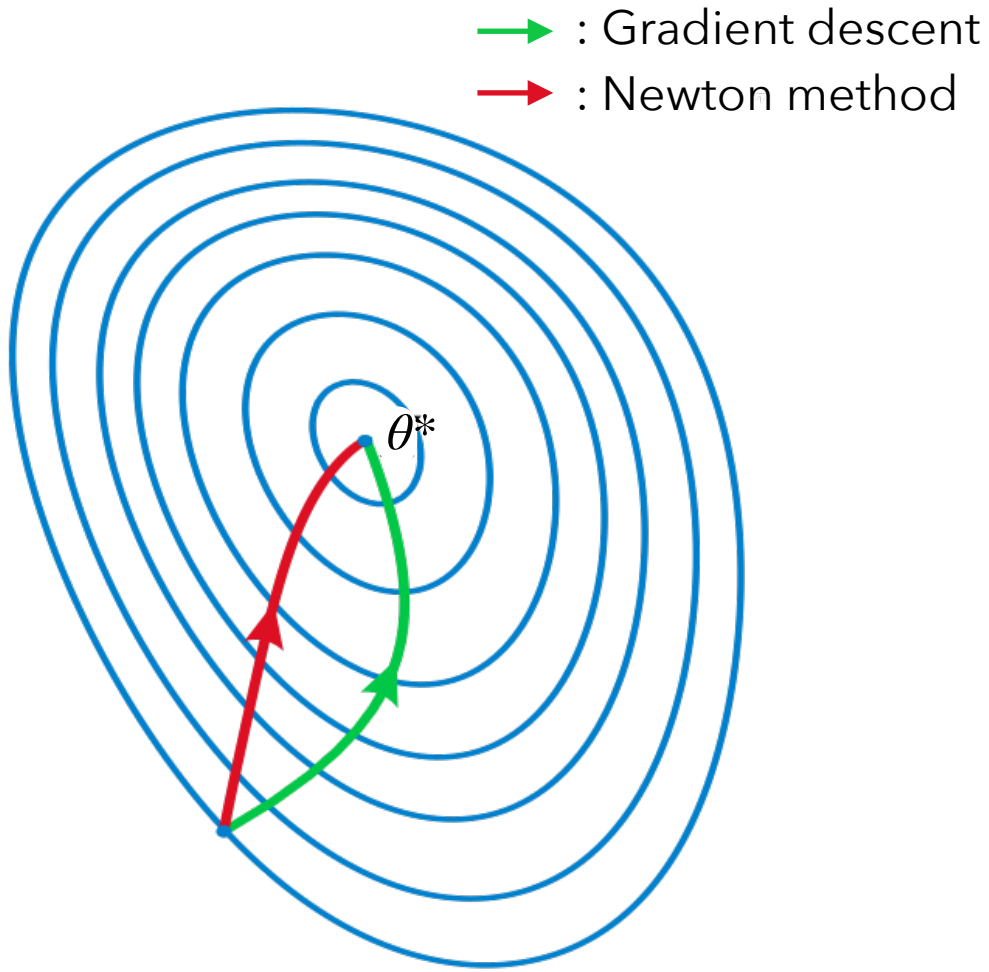
Newton Method - Visualization in Secant Perspective



Optimization

Newton Method - Algorithm

1. **Initialization**: choose an initial guess θ_0
2. **Repeat** until convergence:
   - Compute gradient: $\nabla \mathscr{L}(\theta_t)$
   - Compute Hessian: $H_{\mathscr{L}}(\theta_t)$
   - Update parameters: $\theta_{t+1} = \theta_t - \eta \cdot H_{\mathscr{L}}(\theta_t)^{-1} \nabla \mathscr{L}(\theta_t)$
3. **Convergence check**: if $\|\theta_{t+1} - \theta_t\|$ is small enough, or max iterations reached, then stop
4. **Output**: final parameter θ^* is the optimized solution



Optimization

Quasi-Newton Method - Without Computing Full Hessian

- **Newton method update rule**:
  - $\theta_{t+1} = \theta_t - H_{\mathscr{L}}(\theta_t)^{-1} \nabla \mathscr{L}(\theta_t)$
- **Quasi-Newton method update rule**:
  - $\theta_{t+1} = \theta_t - \alpha_t B_t^{-1} \nabla \mathscr{L}(\theta_t)$
  - Approximate the Hessian matrix with a positive-definite matrix B
  - α_t is a line search parameter
- **Characteristics**:
  - Avoid explicit Hessian matrix computation
  - Uses gradient history for efficiency
  - Achieves near Newton convergence with lower cost

$$H_{\mathscr{L}} = \begin{bmatrix} \frac{\partial^2 \mathscr{L}}{\partial (\theta^0)^2} & \frac{\partial^2 \mathscr{L}}{\partial \theta^0 \partial \theta^1} & \cdots & \frac{\partial^2 \mathscr{L}}{\partial \theta^0 \partial \theta^d} \\ \frac{\partial^2 \mathscr{L}}{\partial \theta^1 \partial \theta^0} & \frac{\partial^2 \mathscr{L}}{\partial (\theta^1)^2} & \cdots & \frac{\partial^2 \mathscr{L}}{\partial \theta^1 \partial \theta^d} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 \mathscr{L}}{\partial \theta^d \partial \theta^0} & \frac{\partial^2 \mathscr{L}}{\partial \theta^d \partial \theta^1} & \cdots & \frac{\partial^2 \mathscr{L}}{\partial (\theta^d)^2} \end{bmatrix}$$

- Computationally expensive
- Not guaranteed positive-definite

Outline

- Loss function
- Optimization
- **Recap: Basic classification**
- Advanced classification

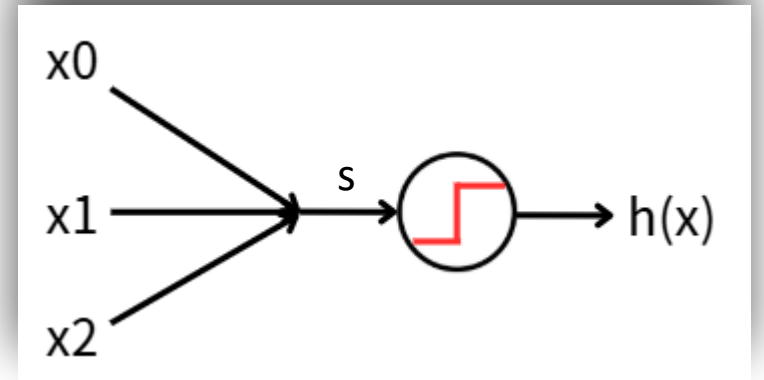


Linear Classifier

Linear Classifier

A fundamental binary classification algorithm that separates data points using a linear decision boundary based on weighted feature combinations

- Linear scoring function :
  $s = \theta^T x + \theta_0 = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d$
- $h(x) = \text{sign}(s) = \text{sign}(\theta^T x + \theta_0)$
- $\text{err}(h, x, y) = I[h(x) \neq y]$



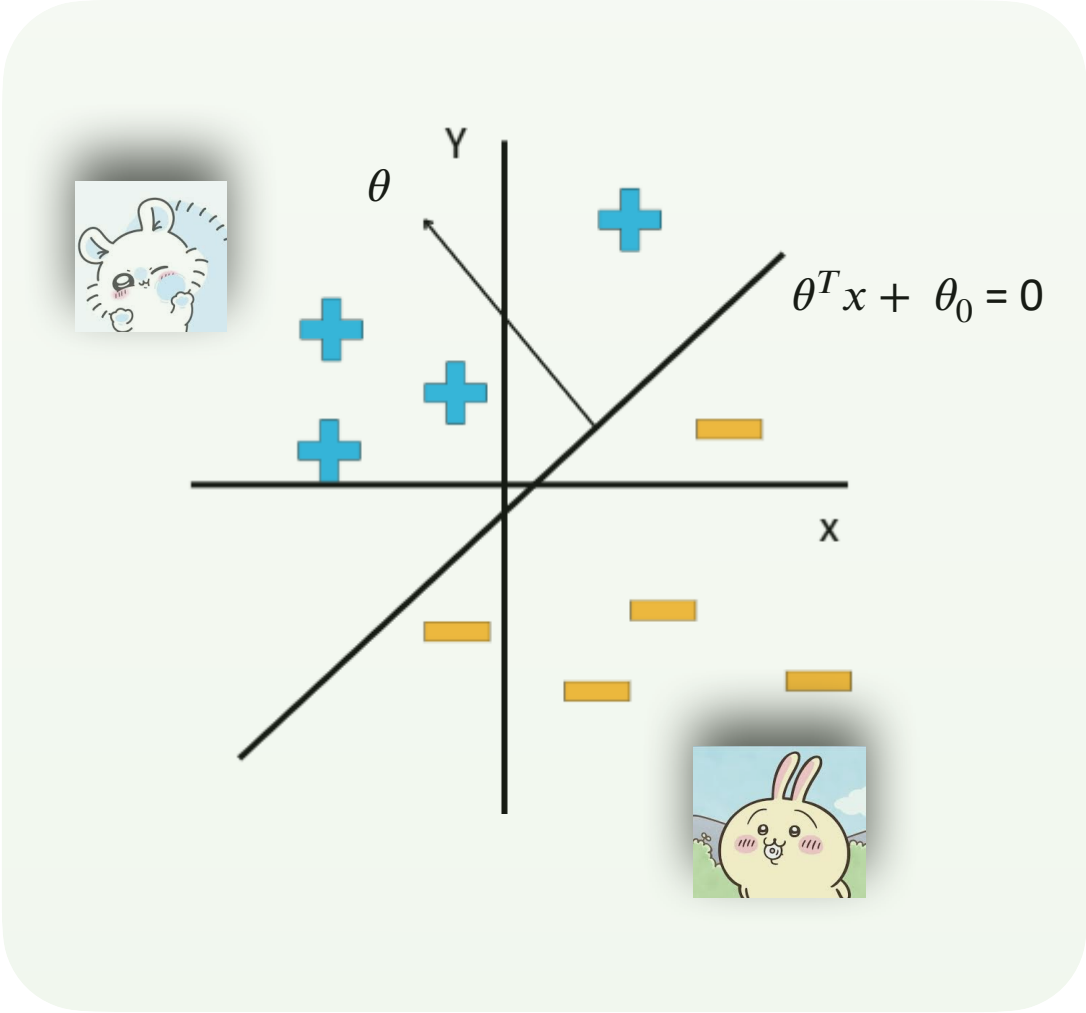
0-1 Loss function:

- $\text{err}_{0/1}(s, y) = I[\text{sign}(s) \neq y]$
- Decision boundary: The hyperplane $\theta^T x + \theta_0$ separates the two classes
- θ is the normal vector to the decision boundary
- $|s|/||\theta||$ gives the distance from point x to the decision boundary

- Linear classifiers learn a hyperplane that optimally divides the feature space into two regions, assigning different class labels to each side
- The decision is made by computing a linear combination of input features and applying a threshold.

Linear Classifier

Random Linear Classifier



$$h(x;\theta,\ \theta_0) = sign(\theta^T x + \theta_0) = \begin{cases} +1, & if\ \theta^T x + \theta_0 \\ -1, & otherwise \end{cases}$$

$$\mathscr{D}_n = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$$

d = number of features/dimensions in input space

$$\epsilon_n(\theta, \theta_0) = \frac{1}{n}\sum_{i=1}^{n} I[y_i \neq \text{sign}(\theta^T x_i + \theta_0)]$$

RANDOM-LINEAR-CLASSIFIER($\mathscr{D}_n, k, d$)

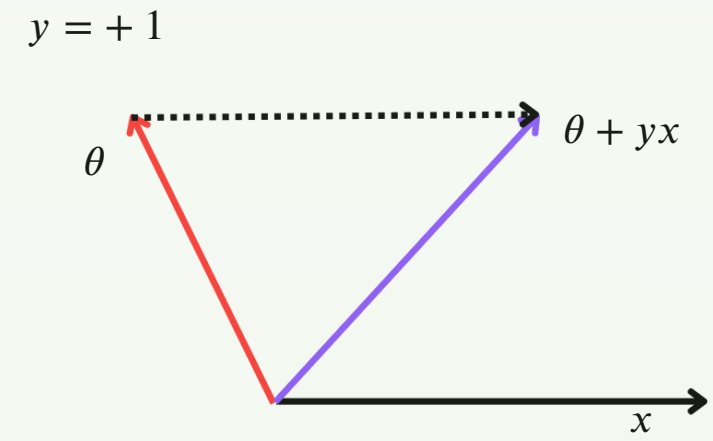
1. For $j = 1$ to k
2. Randomly sample $(\theta^{(j)}, \theta_0^{(j)})$ from $(\mathbb{R}^d, \mathbb{R})$
3. $j^* = \arg\min_{j \in \{1,\ldots,k\}} \epsilon_n(\theta^{(j)}, \theta_0^{(j)})$
4. return $(\theta^{(j^*)}, \theta_0^{(j^*)})$

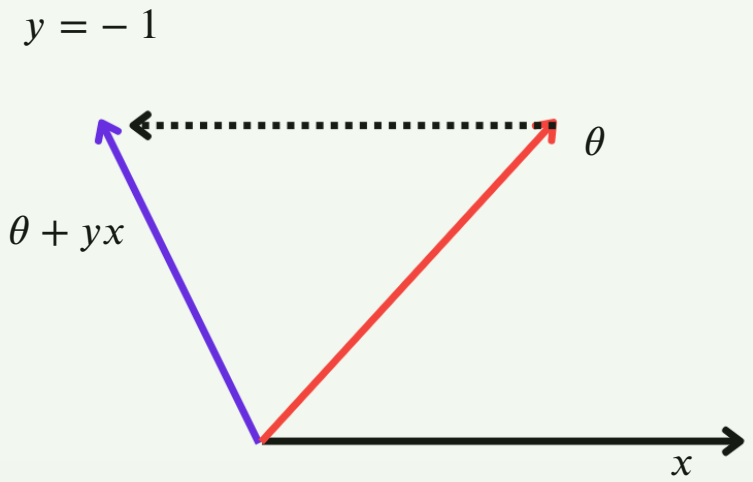
Perceptron Learning

Basic Machine Learning Algorithm

An iterative algorithm that learns linear decision boundaries by correcting misclassification errors

1. **Find a mistake**
   - Misclassified point: $(x_{n(t)}, y_{n(t)})$
   - Condition: $\text{sign}(\theta_t^T x_{n(t)}) \neq y_{n(t)}$
2. **Correct the mistake**
   - $\theta_{t+1} = \theta_t + y_{n(t)} x_{n(t)}$
3. **Repeat until**
   - No mistakes occur, or
   - A stopping condition is reached (e.g. max iterations)





Outline

- Loss function
- Optimization
- Recap: Basic classification
- **Advanced classification**



Linear Support Vector Machine

The Binary Classification Problem

Challenge

Infinitely many separating lines exist!

Key Question:

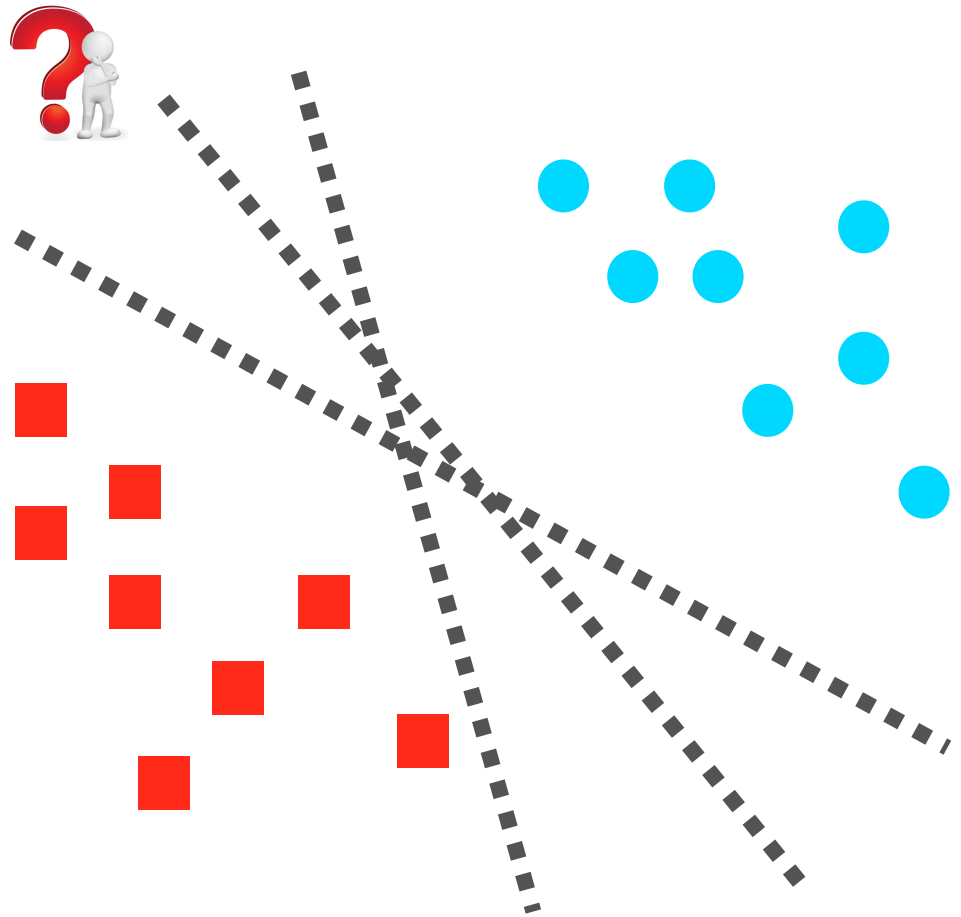
Which line is the best?

Training data

$$\{(\mathbf{x}_i, y_i)\}_{i=1}^{N} \quad \text{where} \quad \$y_i \in \{-1, +1\}$$

Decision function

$$f(\mathbf{x}) = \text{sign}(\boldsymbol{\theta}^T \mathbf{x} + b)$$



Find θ and b !

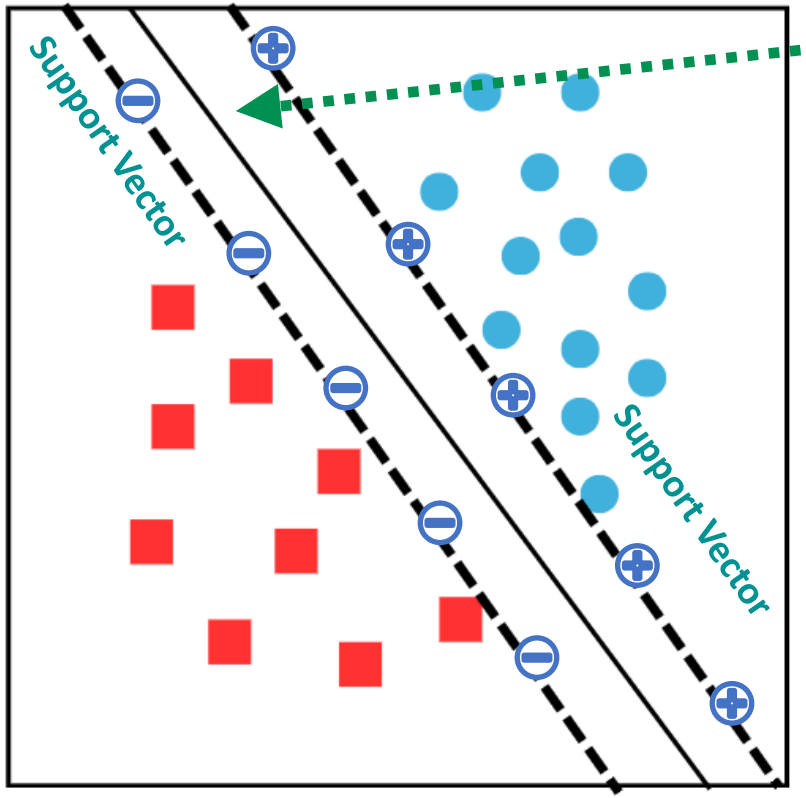
Linear Support Vector Machine

Core Concept: Maximum Margin Classification

Goal: Find the optimal separating hyperplane that maximizes the margin between classes

$$\min_{\theta,b} \frac{1}{2}\|\theta\|^2 \quad \text{subject to} \quad y_i(\theta^T x_i + b) \geq 1, \quad i = 1,2,\ldots,N \quad \text{(indices of the training data points)}$$

Large margin



Hyperplane

$\theta^T x + b = 0$

Dashed lines

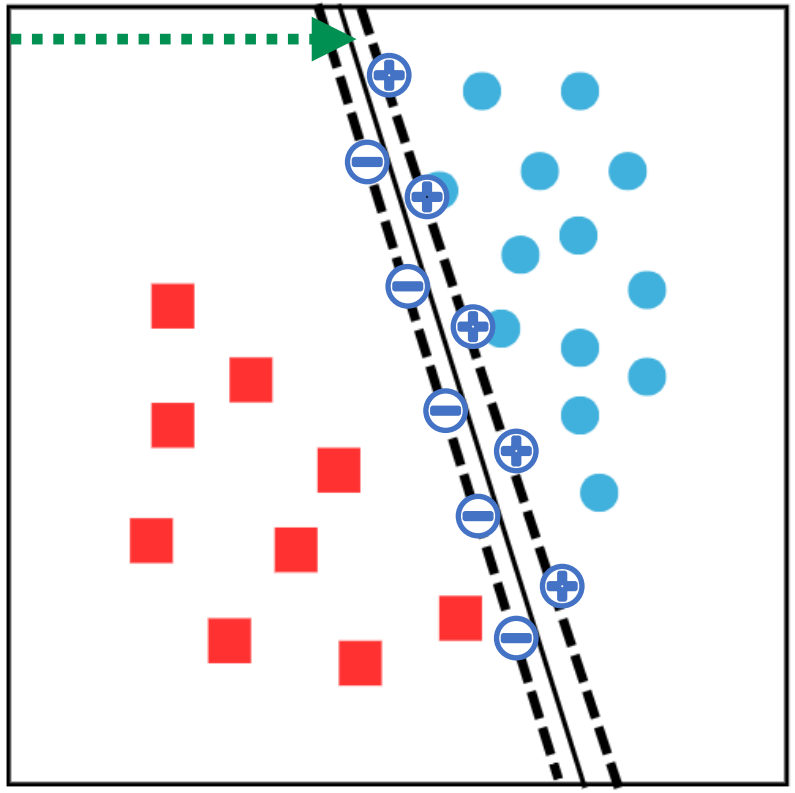
$\theta^T x + b = \pm 1$

Margin

$\text{margin} = \frac{2}{\|\theta\|}$

Decision Rule

$\text{sign}(\theta^T x + b)$



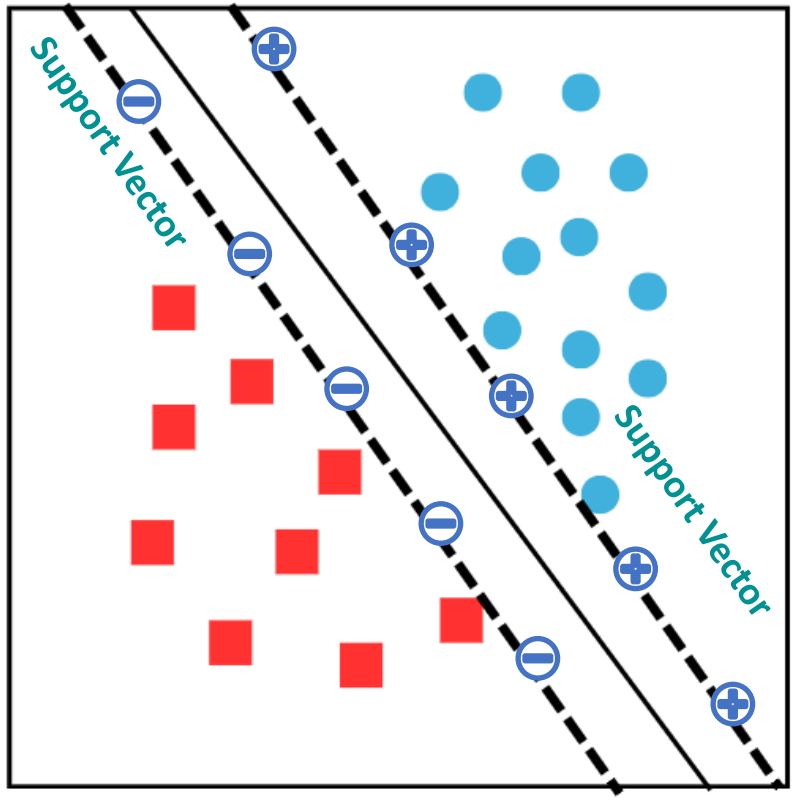
Small margin

Linear Support Vector Machine

The Advantages of Maximum Margin

Goal: Find the optimal separating hyperplane that maximizes the margin between classes

$$\min_{\theta,b} \frac{1}{2}\|\theta\|^2 \quad \text{subject to} \quad y_i(\theta^T x_i + b) \geq 1, \quad i = 1,2,\ldots,N \quad \text{(indices of the training data points)}$$



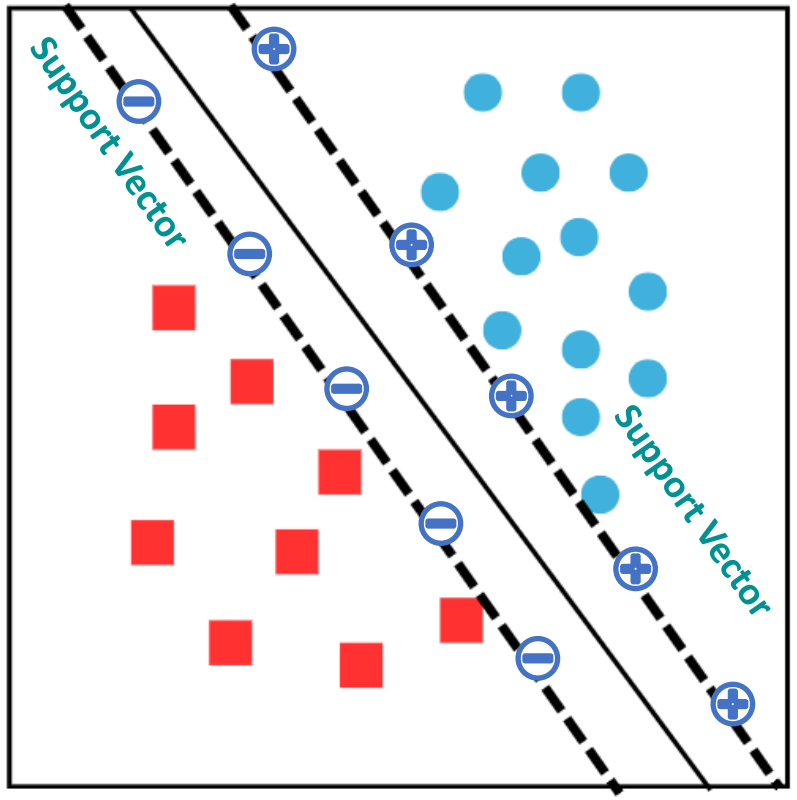
- **Why Maximum Margin?**
  - **Generalization**: Larger margins typically lead to better generalization to **new data**
  - **Robustness**: Maximum margin is least sensitive to small perturbations -> more **robust** to noise
  - **Unique Solution**: Quadratic optimization ensures global optimum (**strictly convex**)

Linear Support Vector Machine

The Concept of Support Vectors

Goal: Find the optimal separating hyperplane that maximizes the margin between classes

$$\min_{\theta,b} \frac{1}{2}\|\theta\|^2 \quad \text{subject to} \quad y_i(\theta^T x_i + b) \geq 1, \quad i = 1,2,\ldots,N \quad \text{(indices of the training data points)}$$



- **Support vector property**
  - Only the closest points (support vectors) determine the optimal hyperplane
  - Lie exactly on the margin boundary
  - Define the optimal hyperplane completely
  - If removed → **different solution**
  - If non-support vectors removed → **same solution**
- **Key concept**
  - Most training points are **irrelevant** → only support vectors matter for the **final model**

Support Vector Machine

SVM Comparisons

Primal SVM

Find optimal hyperplane by minimizing $||\theta||^2$

Problem

Hard optimization in high dimensions

Dual SVM

Transform to dual problem using Lagrange multipliers

Solution

Easier optimization + reveals support vectors

Problem

Still limited to linear boundaries

Kernel SVM

Use kernel trick to handle non-linear data

Solution

Non-linear decision boundaries via $\phi(x)$

Problem

Still assumes perfect separability

Soft-Margin SVM

Allow some misclassification with slack variables

Solution

Handles overlapping classes + noise

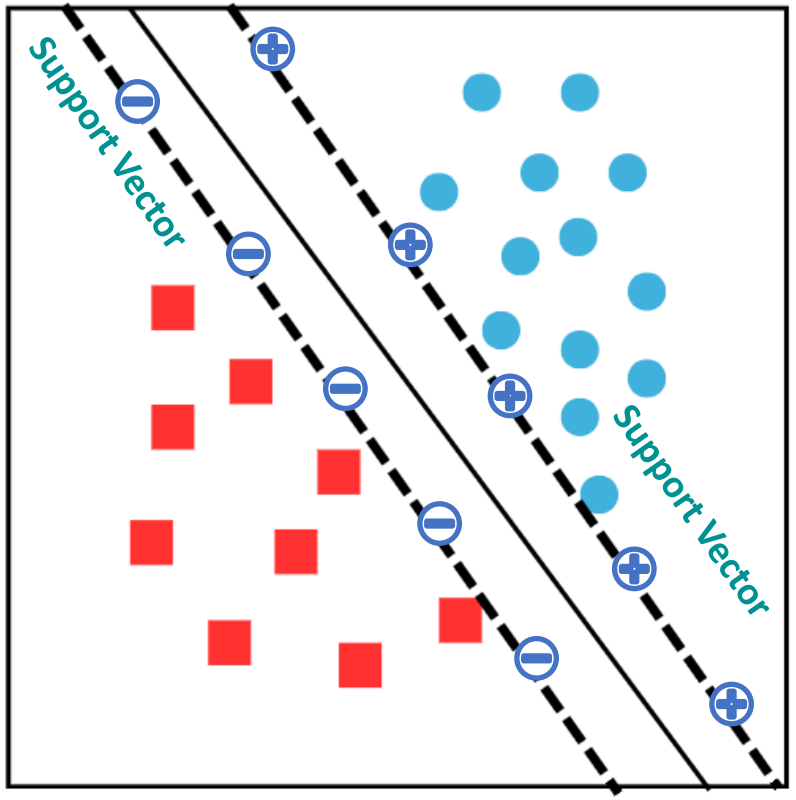
Linear Support Vector Machine

Primal Problem - The Intuitive Goal

Goal: Find the optimal separating hyperplane that maximizes the margin between classes

$$\min_{\theta,b} \frac{1}{2}\|\theta\|^2 \quad \text{subject to} \quad y_i(\theta^T x_i + b) \geq 1, \quad i = 1,2,\ldots,N \quad \text{(indices of the training data points)}$$

- **Key Insight**
  - The margin width is inversely proportional to $\|\theta\|$
  - Larger $\|\theta\|$ → narrower margin
  - Smaller $\|\theta\|$ → wider margin
- **The Optimization Goal**
  - Maximize margin → Minimize $\|\theta\|^2$
    - Becomes a constrained **quadratic optimization** problem
  - We want the widest possible margin
    - **Subject to**: correctly classify all points



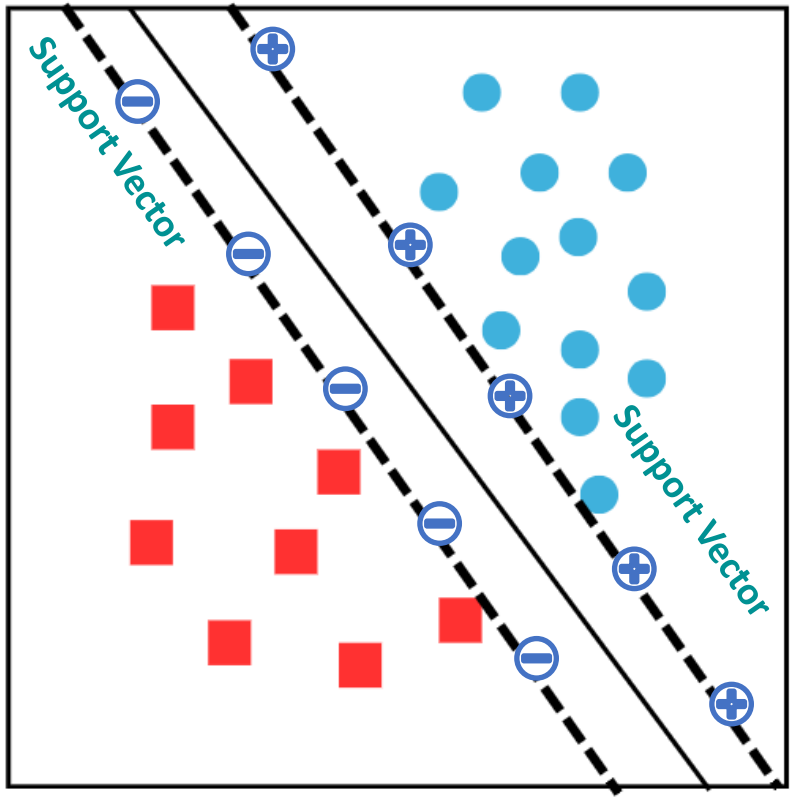
Linear Support Vector Machine

Primal Formulation

Goal: Find the optimal separating hyperplane that maximizes the margin between classes

$$\min_{\theta,b} \frac{1}{2}\|\theta\|^2 \quad \text{subject to} \quad y_i(\theta^T x_i + b) \geq 1, \quad i = 1,2,\ldots,N \quad \text{(indices of the training data points)}$$

- **The Optimization Goal**
  - $1/2\|\theta\|^2$ - Minimize to maximize margin
    - **Constraints**: $y_i(\theta^T x_i + b) \geq 1, \forall i$ - Correct classification with margin
    - **Interpretation**
      - If $y_i = +1, \theta^T x_i + b \geq 1$
      - If $y_i = -1, \theta^T x_i + b \leq 1$
  - This ensures each point is on the correct side of the hyperplane with at least **unit margin**



Linear Support Vector Machine

From Geometric Margin to Hard-Margin SVM

Goal: Find the optimal separating hyperplane that maximizes the margin between classes

$$\min_{\theta,b} \frac{1}{2}\|\theta\|^2 \quad \text{subject to} \quad y_i(\theta^T x_i + b) \geq 1, \quad i = 1,2,\ldots,N \quad \text{(indices of the training data points)}$$

1. **Start from geometric margin** (definition):
   Goal: Find a hyperplane that maximizes the minimum distance to any point

   $$\max_{\theta,b}\{\text{margin}(\theta,b) = \min_{i=1,\ldots,N} \text{distance}(x_i, \text{hyperplane})\} \text{ subject to: } y_i(\theta^T x_i + b) > 0, \quad \forall i$$

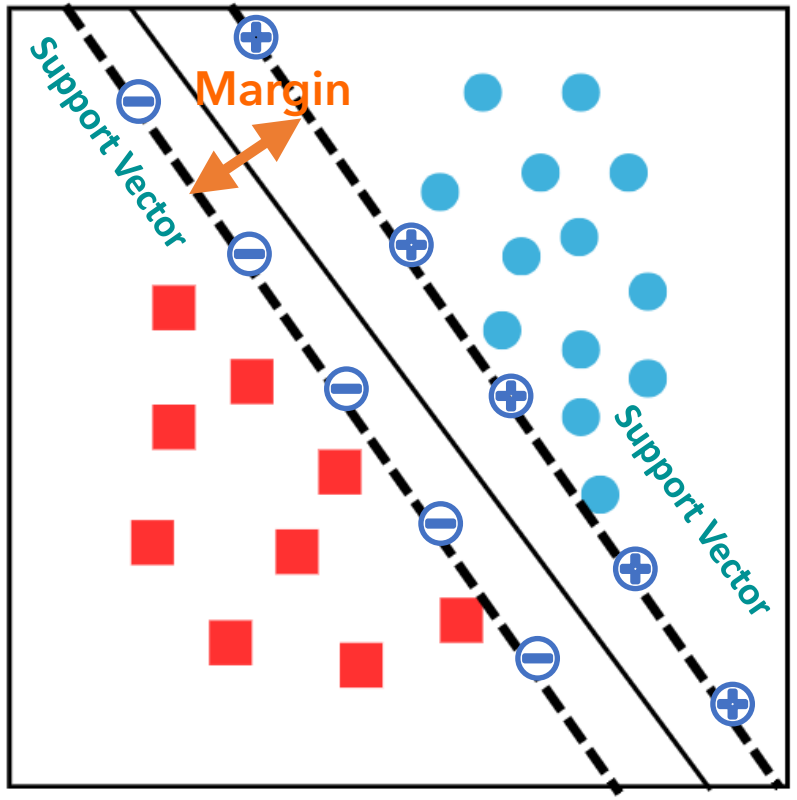
2. **Calculate Distance from Point to Hyperplane**

   $$distance(x_i,\theta,b) = |\theta^T x_i + b|/||\theta|| \Rightarrow margin(\theta,b) = min_{i=1,\ldots,N}(y_i(\theta^T x_i + b))/||\theta||$$

3. **Fix the scaling ambiguity** (normalization)
   Let $min_{i=1,\ldots,N} y_n(\theta^T x_i + b) = 1$ $\Rightarrow$ $\max_{\theta,b}(1/||\theta||)$ subject to: $y_i(\theta^T x_i + b) \geq 1, \forall i$

4. **Turn it into a convex quadratic program (QP)**

   $$\min_{\theta,b} \frac{1}{2}\|\theta\|^2 \text{ subject to: } y_i(\theta^T x_i + b) \geq 1, \quad i = 1,2,\ldots,N$$



Linear Support Vector Machine

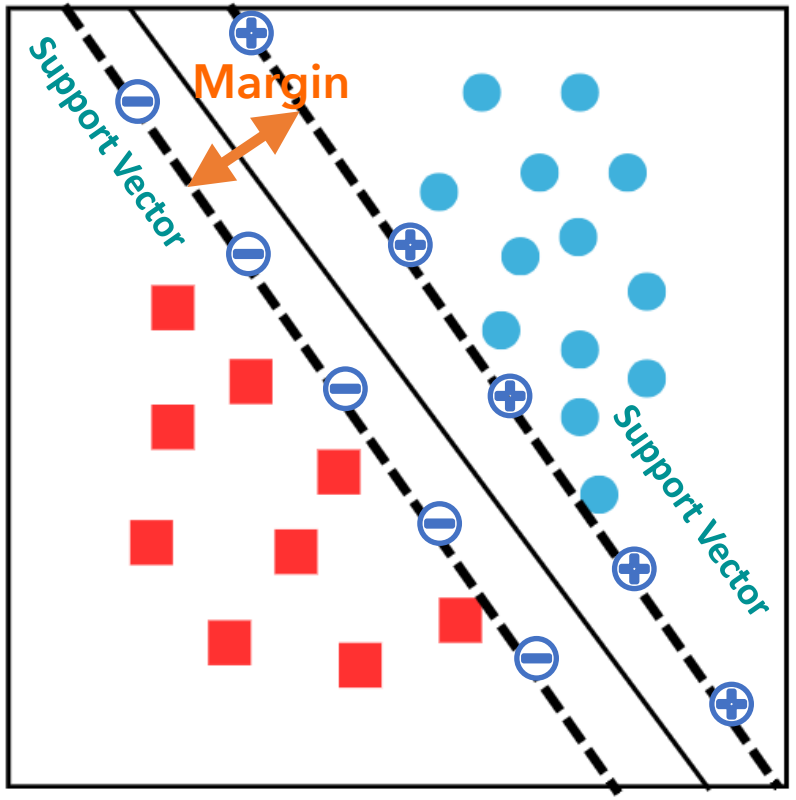
Why Primal is Challenging

Goal: Find the optimal separating hyperplane that maximizes the margin between classes

$$\min_{\theta,b} \frac{1}{2}\|\theta\|^2 \quad \text{subject to} \quad y_i(\theta^T x_i + b) \geq 1, \quad i = 1,2,\ldots,N \quad \text{(indices of the training data points)}$$

Computational Challenges

- **High-dimensional optimization**: If feature dimension d is large, $\boldsymbol{\theta} \in \mathbb{R}^d$ becomes unwieldy
- But which training points are support vectors?
- Primal approach
  - Solve for $\boldsymbol{\theta}^*, b^*$
  - Compute distances for all points
  - Find which distances = **1**
  - **Time complexity** = $O(Nd)$
- Requires additional computation to identify which points lie exactly on the margin boundary, even though this computation is automated in practice

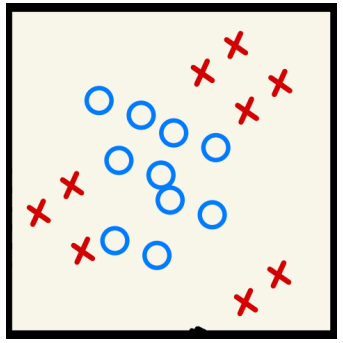


Linear Support Vector Machine

Quadratic Programming (QP)

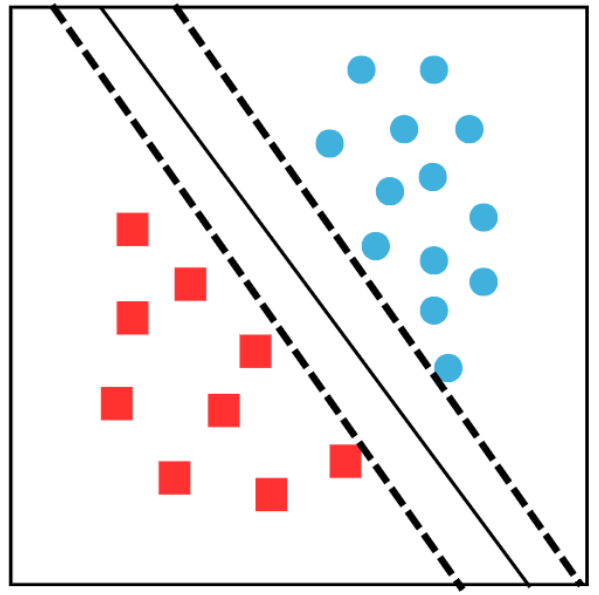
- **The output: $g_{svm}(x) = \text{sign}(\theta^{*T}x + b^*)$ represents the trained SVM classifier function**
  - The final decision function that classifies new data points
  - (θ^*, b^*) are the optimal parameters found by solving either the **primal**optimization problem

- If linearly separable: a linear hyperplane is sufficient : x_i
- Otherwise, enables **more complex hyperplanes** for separation : $z_i = \phi(x_i)$



Linear SVM can't shatter this





$g_{svm}(b, \theta)$

Dual SVM

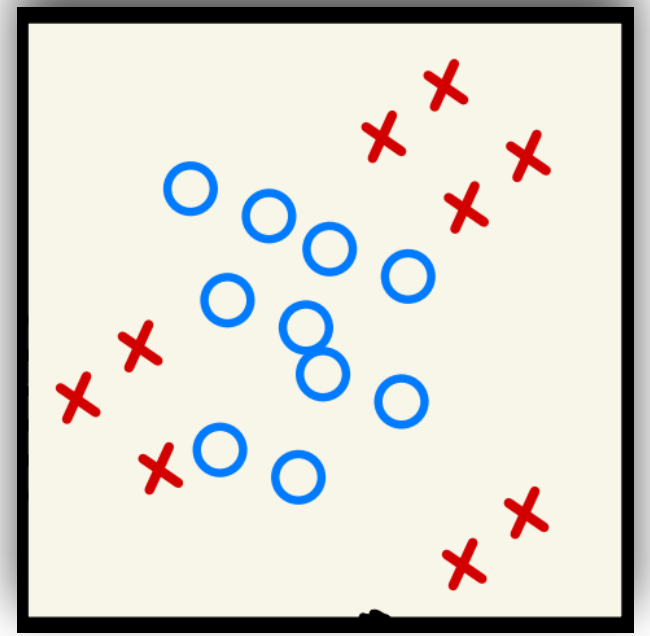
When Data is Mapped to High Dimensions

Non-Linear SVM

- **When data is not linearly separable, we transform features**

$$\min_{b,\theta} \frac{1}{2}\|\theta\|^2 \quad \text{subject to: } y_i(\theta^T z_i + b) \geq 1, \quad \forall i,$$

$$\text{where} \quad z_i = \phi(x_i), \quad x_i \in \mathbb{R}^d \rightarrow z_i \in \mathbb{R}^D$$



- However, when data is in ***high dimension***, it becomes difficult for primal SVM to solve the problem
  - E.g., QP with d features and N constraints ➡ Hard to calculate if d is large even infinite

Objective

- SVM complexity independent of d ➡ Use **Dual SVM**

Linear Support Vector Machine

Primal vs. Dual Problem Formulation

The original problem (Primal Formulation)

- **Find hyperplane parameters directly**
- **Variables**: Hyperplane parameters (θ, b)
- **Objective**: Minimize $\frac{1}{2}\theta^T\theta$ (maximize margin)
- **Constraints**: All points correctly classified with margin ≥ 1
- **Solve for**: Decision boundary $\theta^T x + b = 0$

Total variables: $d(\theta) + 1(b)$

Small features, many samples: Use primal

Dual Problem - Support Vector Optimization

- **Find optimal constraint weights, then compute hyperplane parameters**
- **Variables**: Lagrange multipliers $u = [u_1, u_2, \ldots, u_N]^T$
- **Objective**: Quadratic program $QP(Q, p, A, c)$
- **Constraints**: Non-negativity and label balance conditions
- **Solve for**: **Importance weights** of training points

Total variables: N

Many features, few samples: Use dual

Most modern **SVM libraries** (like **scikit-learn**) automatically choose the best formulation based on your data dimensions

Dual SVM

Derivation - Eliminate Constraints via Lagrange Multipliers

Primal (hard-margin) in feature space

$$\min_{b,\theta} \frac{1}{2}\theta^T\theta \quad \text{s.t. } y_i(\theta^T z_i + b) \geq 1, \quad i = 1,2,\ldots,N$$

For computational reasons with high-dimensional data, we solve an equivalent dual problem

1. Introduce Lagrange multipliers $u_i \geq 0$ (constrained form)

$$\mathcal{L}(b,\theta,u) = \frac{1}{2}\theta^T\theta + \sum_{i=1}^{N} u_i(1 - y_i(\theta^T z_i + b))$$

2. Saddle-point (min-max) form (unconstrained form)

$$\text{SVM} \equiv \min_{b,\theta}\left(\max_{u_i \geq 0} \mathcal{L}(b,\theta,u)\right)$$

3. Lagrange dual problem (mathematical fact)

$$\min_{b,\theta}\left(\max_{u_i \geq 0} \mathcal{L}(b,\theta,u)\right) \geq \max_{u_i \geq 0}\left(\min_{b,\theta} \mathcal{L}(b,\theta,u)\right)$$

Since SVM satisfies all conditions, we get equality $\min_{b,\theta}\max_{\alpha_i \geq 0} \mathcal{L}(b,\theta,\alpha) = \max_{\alpha_i \geq 0}\min_{b,\theta} \mathcal{L}(b,\theta,\alpha)$

Total variables: $d(\theta) + 1(b)$

Small features, many samples: Use primal

'≥' : weakly duality

'=' : strong duality, true for QP if

- Convex primal
- Feasible primal (true if ϕ-separable)
- Linear constraints

Dual SVM

The Lagrange Multipliers (Transform to Unconstrained)

Primal (hard-margin) in feature space

$$\min_{b,\theta} \frac{1}{2}\theta^T\theta \quad \text{s.t. } y_i(\theta^T z_i + b) \geq 1, \quad i = 1,2,\ldots,N$$

For computational reasons with high-dimensional data, we solve an equivalent dual problem

1. Introduce Lagrange multipliers $u_i \geq 0$ (constrained form)

$$\mathcal{L}(b,\theta,u) = \frac{1}{2}\theta^T\theta + \sum_{i=1}^{N} u_i(1 - y_i(\theta^T z_i + b))$$

- **The Trick**
  - Instead of constraints, we add "**penalty terms**" with weights u_i (Lagrange multipliers)
  - Each $u_i \geq 0$ penalizes violations of constraint i

- **Intuition**
  - If constraint i is violated $(y_i(\theta^T z_i + b) < 1)$, then $(1 - y_i(\theta^T z_i + b)) > 0$, so u_i adds a **penalty** to the objective.

Dual SVM

The Saddle Point Problem (Min-Max)

2. Saddle-point (min–max) form

(unconstrained form)

$$\text{SVM} \equiv \min_{b,\theta} \left(\max_{u_i \geq 0} \mathscr{L}(b, \theta, u) \right)$$

- **Key Insight**
  - The original constrained problem is equivalent to: minimizing over (θ, b), while maximizing over $u_i \geq 0$

- **Primal as Min-Max**
  - If constraints **violated:** $(1 - y_i(\theta^T z_i + b)) > 0$
    - u_i can be made arbitrarily large → $\mathscr{L} \to \infty$
  - If constraints **satisfied:** $(1 - y_i(\theta^T z_i + b)) \leq 0$
    - Max over u_i gives $u_i = 0$ → $\mathscr{L} = 1/2\|\theta\|^2$

Dual SVM

Apply Duality Theory (Swap Min and Max)

3. Lagrange dual problem

(mathematical fact)

$$\min_{b,\theta}\left(\max_{u_i\geq 0}\mathscr{L}(b,\theta,u)\right)\geq\max_{u_i\geq 0}\left(\min_{b,\theta}\mathscr{L}(b,\theta,u)\right)$$

Primal Form Dual Form

- **Mathematical Fact**
  - For certain problems, we can swap the order of min and max operations. This gives us the dual problem

- **Relationship**
  - Left side ≥ Right side (**weak duality**)
  - **Strong Duality Conditions** (When Equality Holds) ✅
  - For SVM, we get **EQUALITY** (strong duality)
    - **Convex primal**: $1/2\|\theta\|^2$ is convex quadratic
    - **Feasible primal**: **Linearly separable data** (solution exists)
    - **Linear constraints**: $y_i(\theta^T z_i + b) \geq 1$ are linear in (θ, b)

Dual SVM

Derivation of Dual SVM: How do We Solve It?

1. Primal with Lagrange

$$\max_{u_i \geq 0} \underbrace{\left(\min_{b,\theta} \frac{1}{2}\theta^T\theta + \sum_{i=1}^{N} u_i(1 - y_i(\theta^T z_i + b)) \right)}_{\mathscr{L}(b,\theta,u)}$$

2. Expansion

- $\mathscr{L}(b,\theta,u) = \frac{1}{2}\theta^T\theta + \sum_{i=1}^{N} u_i - \sum_{i=1}^{N} u_i y_i \theta^T z_i - \sum_{i=1}^{N} u_i y_i b$
- $\mathscr{L}(b,\theta,u) = \frac{1}{2}\theta^T\theta - \theta^T\left(\sum_{i=1}^{N} u_i y_i z_i\right) - b\left(\sum_{i=1}^{N} u_i y_i\right) + \sum_{i=1}^{N} u_i$
- $\max_{u_i \geq 0}\left(\min_{b,\theta}\left[\frac{1}{2}\theta^T\theta - \theta^T\left(\sum_{i=1}^{N} u_i y_i z_i\right) - b\left(\sum_{i=1}^{N} u_i y_i\right) + \sum_{i=1}^{N} u_i\right]\right)$

3. Take derivative with respect to b and set to zero:

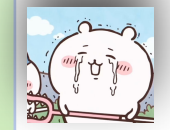
- $\frac{\partial \mathscr{L}}{\partial b} = 0 = -\sum_{i=1}^{N} u_i y_i \quad \Rightarrow \sum_{i=1}^{N} u_i y_i = 0$

4. Take derivative with respect to θ and set to zero:

- $\frac{\partial \mathscr{L}}{\partial \theta} = 0 = \theta - \sum_{i=1}^{N} u_i y_i z_i \quad \Rightarrow \theta = \sum_{i=1}^{N} u_i y_i z_i$

5. Final dual problem

- $\max_{\substack{u_i \geq 0 \\ \sum_i u_i y_i = 0}} \left[-\frac{1}{2} \left\| \sum_{i=1}^{N} u_i y_i z_i \right\|^2 + \sum_{i=1}^{N} u_i \right]$



Dual SVM

KKT (Karush-Kuhn-Tucker) Optimality Conditions

$$\max_{\substack{u_i \geq 0 \\ \sum_i y_i u_i = 0 \\ \theta = \sum_i u_i y_i z_i}} \left[-\frac{1}{2} \left\| \sum_{i=1}^{N} u_i y_i z_i \right\|^2 + \sum_{i=1}^{N} u_i \right]$$

- If primal-dual optimal solutions (θ^*, b^*, u^*)
- **Primal** feasible: $y_i(\theta^T z_i + b) \geq 1, \quad \forall i$
- **Dual** feasible: $u_i \geq 0, \quad \forall i$
- **Dual**-inner optimal: $\sum_{i=1}^{N} y_i u_i = 0, \quad \theta = \sum_{i=1}^{N} u_i y_i z_i$
- **Primal**-inner optimal (at optimal all **Lagrange terms** disappear)

$$u_i(1 - y_i(\theta^T z_i + b)) = 0, \quad i = 1,2,\ldots,N$$

Dual SVM

Quadratic Programming

Standard hard-margin SVM dual problem (min form)

$$\max_{\substack{u_i \geq 0 \\ \sum_i y_i u_i = 0 \\ \theta = \sum_i u_i y_i z_i}} \left[-\frac{1}{2} \left\| \sum_{i=1}^{N} u_i y_i z_i \right\|^2 + \sum_{i=1}^{N} u_i \right]$$



- $$\left\| \sum_{i=1}^{N} u_i y_i z_i \right\|^2 = \left(\sum_{i=1}^{N} u_i y_i z_i \right)^T \left(\sum_{i=1}^{N} u_i y_i z_i \right) = \sum_{i=1}^{N} \sum_{j=1}^{N} u_i u_j y_i y_j z_i^T z_j$$

Standard hard-margin SVM dual problem (min form): $\min_u \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} u_i u_j y_i y_j z_i^T z_j - \sum_{i=1}^{N} u_i \quad \text{s.t.} \sum_{i=1}^{N} y_i u_i = 0; \quad u_i \geq 0, \forall i$

Standard QP form: $u^* = \arg\min_u \left\{ \frac{1}{2} u^T Q u + p^T u : A^T u = b, u \geq 0 \right\}$, for $i = 1,2,\ldots,$ where $Q_{ij} = y_i y_j z_i^T z_j, p = \begin{bmatrix} -1 \\ -1 \\ \vdots \\ -1 \end{bmatrix}$

- Use **QP solvers** to find the optimal u^*

$$\theta^* = \sum_{i=1}^{N} u_i^* y_i z_i \qquad b^* = \frac{1}{|SV|} \sum_{i \in SV} (y_i - \theta^{*T} z_i), \quad \text{where } SV = \{i : u_i^* > \epsilon\}$$

Dual SVM

Support Vector

$u_i^* > 0$: **On boundary (most $u_i = 0$ (sparse solution))**

- Only SV needed to compute θ:

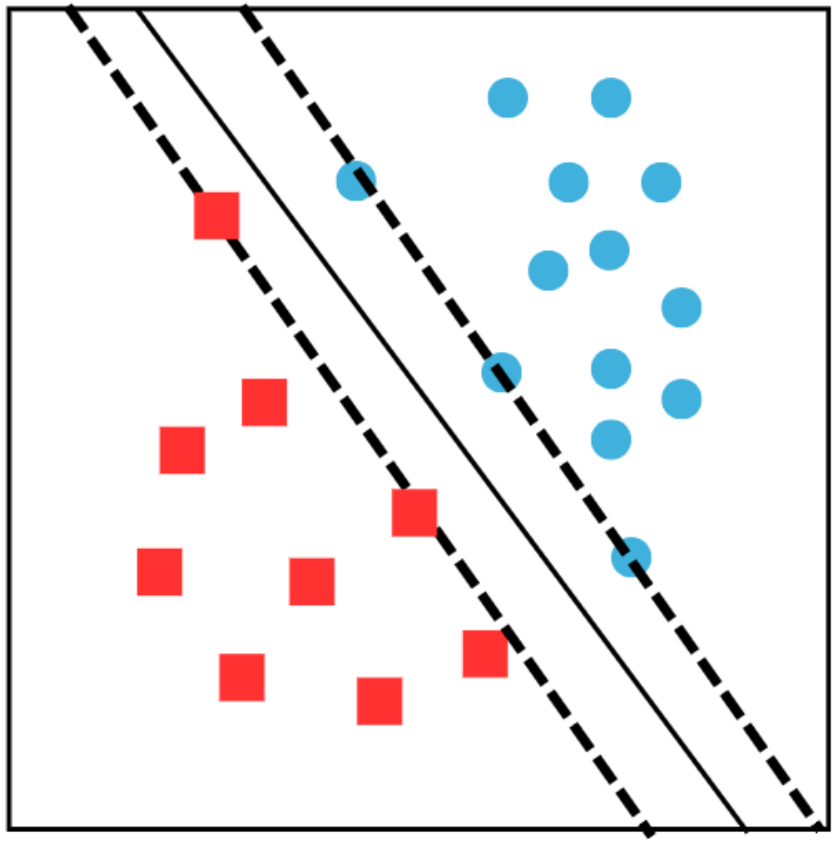
$$\theta = \sum_{i \in SV} u_i y_i z_i$$

- Only SV needed to compute b:

$$b = y_i - \theta^T z_i \quad \text{with any } SV(z_i, y_i)$$

$$u_i^* > 0 \Leftrightarrow y_i(\theta^{*T} z_i + b^*) = 1 \Leftrightarrow \text{point } i \text{ on margin boundary}$$

Decision boundary: $f(x) = \text{sign}(\theta^{*T} \phi(x) + b^*)$





$u_i > 0$
support vector

SVM

Comparison of Primal SVM & Dual SVM

Primal Hard-Margin SVM

- $\min_{b,\theta} \frac{1}{2}\theta^T\theta \quad \text{s.t. } y_i(\theta^T x_i + b) \geq 1, \quad i = 1,\ldots,N$
- Locate specially-scaled (θ, b)

Dual Hard-Margin SVM

- $\min_{u} \frac{1}{2}u^T Q u - 1^T u \quad \text{s.t. } y^T u = 0; \quad u_i \geq 0, \quad i = 1,\ldots,N$
- $\min_{u} \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} u_i u_j y_i y_j x_i^T x_j - \sum_{i=1}^{N} u_i \quad \text{s.t. } \sum_{i=1}^{N} y_i u_i = 0; \quad u_i \geq 0$
- Locate $SVs(z_i, y_i)$ using u_i

Both eventually result in optimal (θ, b) for fattest hyperplane

$$g_{SVM} = \text{sign}(\theta^T \phi(x) + b)$$

Kernel SVM

Kernel Trick Enables Efficient Non-Linear Separation without Explicit High-Dimensional Mapping

The kernel trick makes non-linear SVM computationally feasible

Goal: Working in high-dimensional feature spaces without explicitly computing the transformed features $\phi(x)$

- **Key Insight**
  - **Instead of computing $\phi(x)^T\phi(x')$ (expensive high-dimensional computation)**
  - **Using efficient kernel function $K(x, x')$**

- **Dual SVM:** $\min_u \frac{1}{2}u^TQu - 1^Tu$ **s.t.** $y^Tu = 0;$ $u_i \geq 0$, **for** $i = 1,2,\ldots,N$

  - $Q_{i,j} = y_iy_jK(x_i, x_j)$ **where** $K(x_i, x_j) = \phi(x_i)^T\phi(x_j)$

    $z_i^Tz_j = \phi(x_i)^T\phi(x_j) = K(x_i, x_j)$ **(compute directly, faster)**

- **Kernel functions that directly compute inner products in the transformed space**

Kernel SVM

Example Kernels

Linear Kernel

$$K(x, z) = xz$$

Boundary

Linear hyperplane

Use when

Data is linearly separable

Polynomial Kernel

$$K(x, z) = (\gamma xz + r)^d$$

Boundary

Smooth curves

Parameter

degree d, γ, r

RBF (Gaussian) Kernel

$$K(x, z) = exp(-\gamma \|x - z\|^2)$$

Boundary

γ (bandwidth)

Parameter

γ (bandwidth)

Sigmoid Kernel

$$K(x, z) = tanh(\gamma xz + r)$$

Connection

Neural networks

Parameters

γ, r

Kernel SVM

The Polynomial Kernel Example 🚀

2nd order polynomial transform

$$\phi_2(\mathbf{x}) = (1, x_1, x_2, \ldots, x_d, x_1^2, x_1x_2, \ldots, x_1x_d, x_2x_1, x_2^2, \ldots, x_2x_d, \ldots, x_d^2)$$

- What $\phi_2(\mathbf{x})$ Contains
  - Constant: 1
  - Linear terms: $x_1, x_2, \ldots, x_d$
  - Squared terms: $x_1^2, x_2^2, \ldots, x_d^2$
  - Cross products: $x_1x_2, x_1x_3, \ldots, x_{d-1}x_d$
- $\phi_2(\mathbf{x})$ contains: $\{1\} \cup \{x_i\}_{i=1}^d \cup \{x_i^2\}_{i=1}^d \cup \{x_ix_j\}_{1 \le i < j \le d}$

- Naive way ($O(d^2)$)
  - Compute $\phi_2(\mathbf{x})$ explicitly
  - Compute $\phi_2(\mathbf{x}')$ explicitly
  - Take dot product $\phi(x)^T\phi(x')$
- Smart way (Kernel Trick) ($O(d)$)
  - Never compute $\phi_2(\mathbf{x})$ explicitly

Kernel SVM

The 2nd Order Polynomial Kernel Example 🚀: Expansion

2nd order polynomial transform

$$\phi_2(\mathbf{x}) = (1, x_1, x_2, \ldots, x_d, x_1^2, x_1x_2, \ldots, x_1x_d, x_2x_1, x_2^2, \ldots, x_2x_d, \ldots, x_d^2)$$

- 1. Direct computation (expensive)
  - $\phi_2(\mathbf{x})^T\phi_2(\mathbf{x}') = 1 + \sum_{i=1}^{d} x_i x_i' + \sum_{i=1}^{d}\sum_{j=1}^{d} x_i x_j x_i' x_j'$
- 2. Rearranged form
  - $\phi_2(\mathbf{x})^T\phi_2(\mathbf{x}') = 1 + \sum_{i=1}^{d} x_i x_i' + \sum_{i=1}^{d} x_i x_i' \sum_{j=1}^{d} x_j x_j'$
- 3. Final kernel form
  - $\phi_2(\mathbf{x})^T\phi_2(\mathbf{x}') = 1 + \mathbf{x}^T\mathbf{x}' + (\mathbf{x}^T\mathbf{x}')^2$
- 4. Complexity improvement
  - $O(d^2) \rightarrow O(d)$

Kernel SVM

Plug in Efficient Kernel Function to Avoid Dependence on $\tilde{d}$

Quadratic coefficient

$$q_{i,j} = y_i y_j \mathbf{z}_i^T \mathbf{z}_j = y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

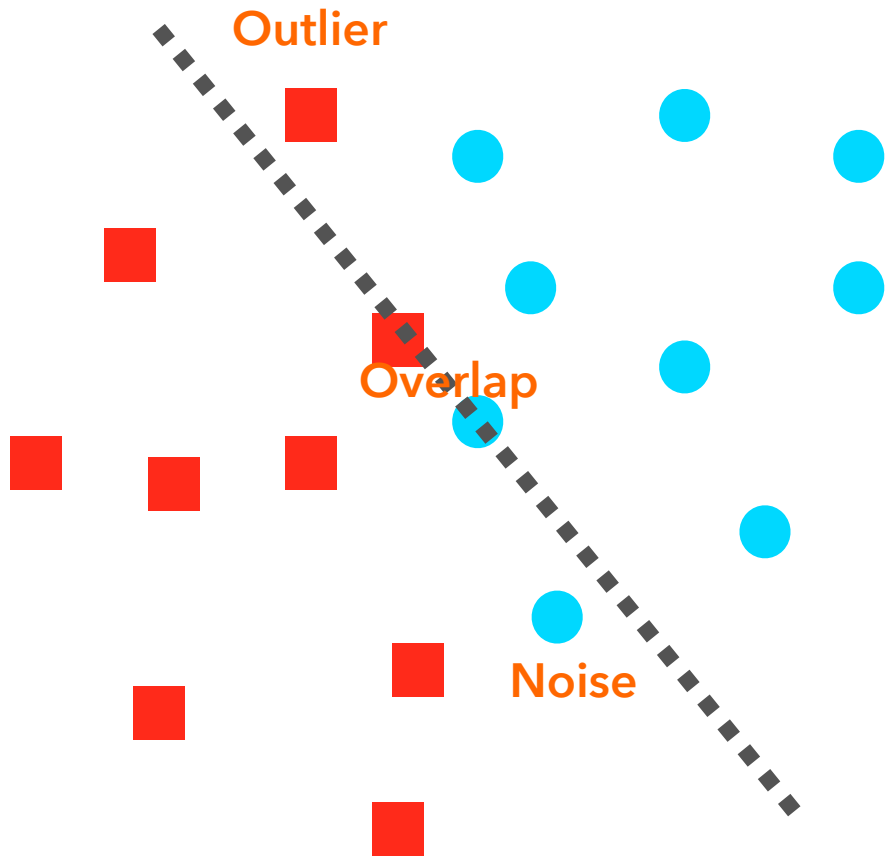
$O(N^2)$ (kernel evaluations)

- 1. Solving the Dual Problem
  - $\mathbf{u} \leftarrow QP$
- 2. Bias Computation
  - $b = y - \boldsymbol{\theta}^T \mathbf{z} = y - \left(\sum_{i=1}^{N} u_i y_i \mathbf{z}_i \right)^T \mathbf{z} = y - \sum_{i=1}^{N} u_i y_i K(\mathbf{x}_i, \mathbf{x})$
- 3. Final Prediction Function
  - $g_{svm}(\mathbf{x}) = \text{sign}(\boldsymbol{\theta}^T \phi(\mathbf{x}) + b) = \text{sign}\left(\sum_{i=1}^{N} u_i y_i K(\mathbf{x}_i, \mathbf{x}) + b \right)$

Soft-Margin SVM

Real-World Reality Check

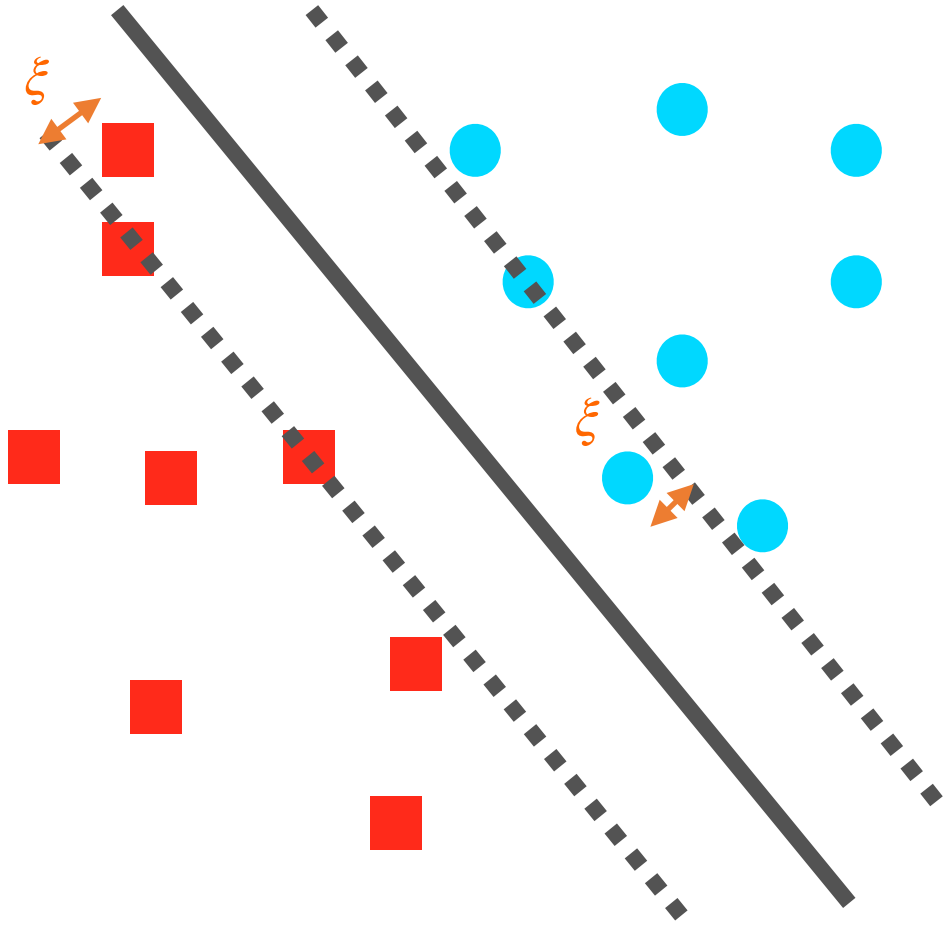
- **Hard-Margin SVM Failures**
  - **Overlapping**: No perfect linear separation exists
  - **Outliers**: Isolated noisy points
  - **Measurement errors**: Mislabeled data
  - **Natural variation**: Real data is messy!
- **The Problem**
  - Hard-margin SVM fails completely when data is not linearly separable. The optimization has no feasible solution!
- **What We Need**
  - A way to allow some "**mistakes**" while still finding the best possible decision boundary



Soft-Margin SVM

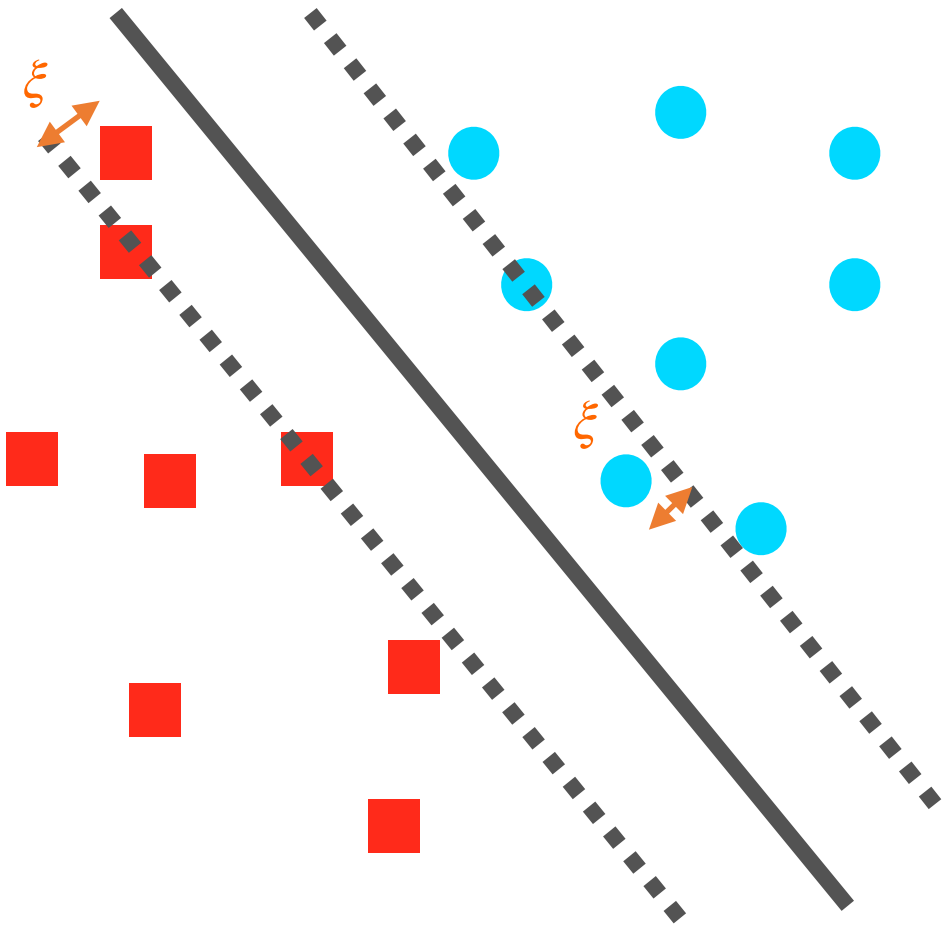
Introduce "Forgiveness" for Points that Can't be Perfectly Separated

- **The Slack Variable Idea**
  - For each training point, introduce a slack variable $\xi \geq 0$ that measures how much the point violates the margin constraint
- **What Slack Variables Allow**
  - $\xi = 0$: Point satisfies margin constraint perfectly
  - $0 < \xi < 1$: Point is correctly classified but inside margin
  - $\xi = 1$: Point is exactly on the decision boundary
  - $\xi > 1$: Point is misclassified (wrong side of boundary)
- **Key Insight**
  - ξ measures the "**amount of violation**" for each point. We'll minimize total violation while maximizing margin



Soft-Margin SVM

Soft-Margin SVM Formulation

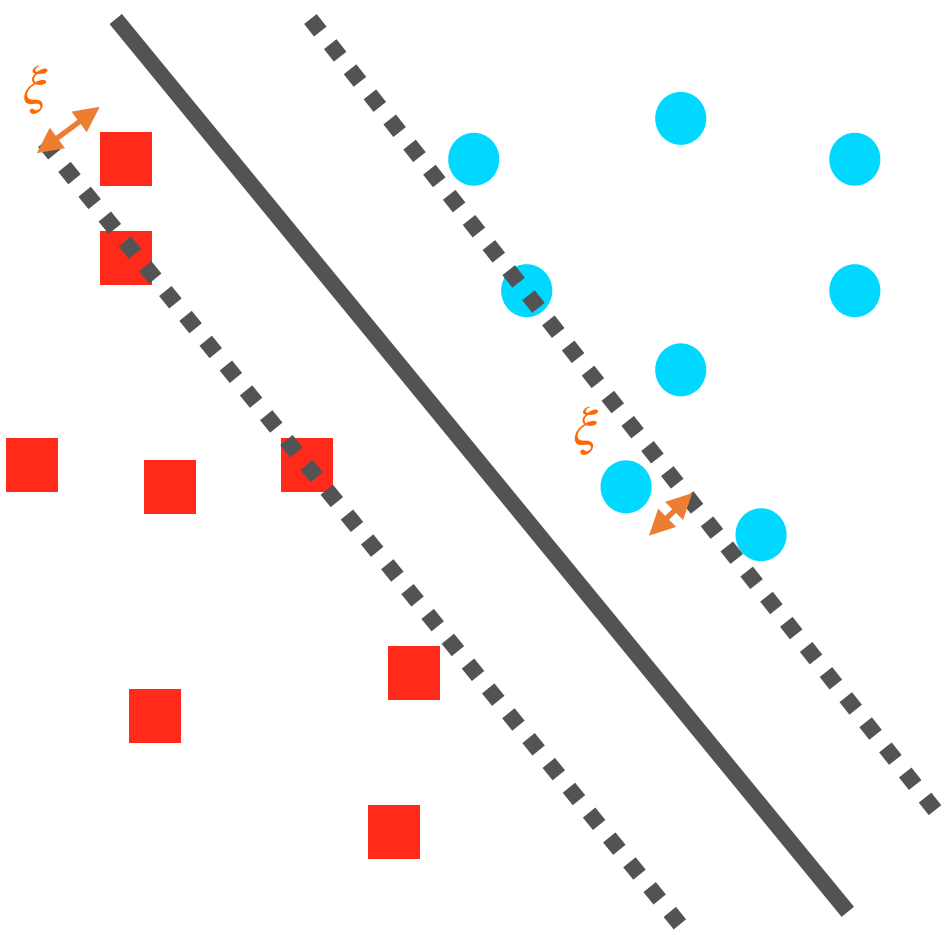


- **Hard-margin SVM**
  - $\min_{\boldsymbol{\theta},b} \frac{1}{2}\boldsymbol{\theta}^T\boldsymbol{\theta}$ s.t. $y_i(\boldsymbol{\theta}^T\mathbf{z}_i + b) \geq 1 \quad \forall i$
- **Problem with hard-margin**
  - If data not separable: $\nexists$ solution to hard constraints
- **Soft-margin SVM with slack variables**
  - $\min_{\boldsymbol{\theta},b,\boldsymbol{\xi}} \frac{1}{2}\boldsymbol{\theta}^T\boldsymbol{\theta} + C\sum_{i=1}^{N}\xi_i$

    s.t. $y_i(\boldsymbol{\theta}^T\mathbf{z}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0 \quad \forall i$
  - **Large C**: Focus on minimizing training errors
  - **Small C**: Focus on maximizing margin

Soft-Margin SVM

Soft-Margin SVM Dual Formulation



- **Hard-margin dual SVM**
  - $\max_{\mathbf{u}} \sum_{i=1}^{n} u_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} u_i u_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$ subject to: $\sum_{i=1}^{n} u_i y_i = 0, \quad u_i \geq 0 \quad \forall i$
- **Soft-Margin Dual**
  - $\max_{\mathbf{u}} \sum_{i=1}^{n} u_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} u_i u_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$ subject to: $\sum_{i=1}^{n} u_i y_i = 0, \quad 0 \leq u_i \leq C \quad \forall i$
- **Key Insight**
  - Only change: $u_i \geq 0 \rightarrow 0 \leq u_i \leq C$

    s.t. $y_i(\boldsymbol{\theta}^T \mathbf{z}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0 \quad \forall i$

Thank you for your attention!

